

CACTUSDB: Unlock Co-Optimization Opportunities for SQL and AI/ML Inferences

Lixi Zhou^a, Kanchan Chowdhury^a, Lulu Xie^a, Jaykumar Tandel^a,
Hong Guan^a, Zhiwei Fan^b, Xinwei Fu^c, Jia Zou^a

^a Arizona State University, ^b Meta, ^c Amazon

Abstract—There is a growing demand for supporting *inference queries* that combine Structured Query Language (SQL) and Artificial Intelligence / Machine Learning (AI/ML) model inferences in database systems, to avoid data denormalization and transfer, facilitate management, and alleviate privacy concerns. Co-optimization techniques for executing inference queries in database systems without accuracy loss fall into four categories: (O1) Relational algebra optimization treating AI/ML models as black-box user-defined functions (UDFs); (O2) Factorized AI/ML inferences; (O3) Tensor-relational transformation; and (O4) General cross-optimization techniques. However, we found none of the existing database systems support all these techniques simultaneously, resulting in suboptimal performance. In this work, we identify two key challenges to address the above problem: (1) the difficulty of unifying all co-optimization techniques that involve disparate data and computation abstractions in one system; and (2) the lack of an optimizer that can effectively explore the exponential search space. To address these challenges, we present CACTUSDB, a novel system built atop Velox - a high-performance, UDF-centric database engine, open-sourced by Meta. CACTUSDB features a three-level Intermediate Representations (IR) that supports relational operators, expression operators, and ML functions to enable flexible optimization of arbitrary sub-computations. Additionally, we propose a novel Monte-Carlo Tree Search (MCTS)-based optimizer with query embedding, co-designed with our unique three-level IR, enabling shared and reusable optimization knowledge across different queries. Evaluation of 12 representative inference workloads and 2,000 randomly generated inference queries on well-known datasets, such as MovieLens and TPCx-AI, shows that CACTUSDB achieves up to 441× speedup compared to alternative systems.

Index Terms—database, machine learning, query optimization

I. INTRODUCTION

It is important to nest Artificial Intelligence / Machine Learning (AI/ML) inferences and Structured Query Language (SQL) queries to drive high-value insights from relational data, e.g., to generate personalized product recommendations [1]–[3] based on preprocessed customer profiles and online shopping records through `join`, `filter`, and `project` nested with AI/ML model inferences, to obtain statistics about credit card transactions that are predicted as fraud by a cheap and less accurate XGBoost model and predicted as non-fraud by an expensive and more accurate DNN model [4], through `aggregate`, `join`, and `filter`, nested with inferences. Therefore, a recent trend of supporting SQL queries nested with AI/ML inference functions within database systems has emerged [5]–[11]. This approach eliminates the need to transfer data from databases to ML systems. It not only reduces latency in workloads where data transfer becomes a bottleneck [12]

but also alleviates privacy concerns [13], [14] and operational overhead [15], [16].

Most importantly, having SQL and ML running in the same system will unlock co-optimization opportunities and reduce execution costs without affecting accuracy. We summarize the co-optimization techniques into four categories:

(O1) Relational algebra optimization. An inference query may involve AI/ML-based filters that invoke AI/ML models on the input data to return a Boolean value, such as `predict(movieFeatures)=True`. Based on the estimated costs of the AI/ML models and the selectivity of the predicates, the system can push down such filters through `join`, without knowing the internal logic of the AI/ML models to avoid redundant computations. Similar techniques include pushing down AI/ML-based `project` operators, reordering two consecutive customized filters, and merging multiple consecutive AI/ML-based filter/projects into one. User-Defined Function (UDF)-centric systems [6], [17] and systems with extended operators [5] support some of these techniques. **(O2) Factorized inference.** An AI workflow (e.g., a two-tower recommendation model [18]) may process features that are joined from multiple tables. These workflows/functions can often be factorized into multiple independent functions, each processing features from a separate table. Then, each factorized computation can be pushed down through the `join` to its corresponding table to avoid redundant computations. The idea was extended from several works [19]–[21] in the context of factorized ML [19], [22].

(O3) Tensor-relational transformation. Model parameters (e.g., weight matrices or decision trees) can be stored as relational tables, and inference can be rewritten into relational operators such as `join/project/aggregate` over these tables. This makes inference transparent to the database engine and allows it to scale via buffer-pool management and pipelined, parallel execution, even when model parameters are too large to fit in memory [12], [23], [24].

(O4) Data-model and cross-library optimization techniques. Seeing AI/ML as a white box, there are more cross-optimization techniques, e.g., fusing AI/ML operators from different libraries [25], [26], pruning nodes from decision forest models based on the relational predicates applied to the data to be inferred, and pruning attributes from datasets based on model logic [27]. Another co-optimization technique in this category is to replace ML operators on dense tensors with operators optimized for sparse tensors, assuming that the database is

aware that the input data is sparse [28].

The Problem and the Challenges. As detailed in Sec. II-B, most of the existing DB-for-AI systems [5]–[7], [17], [20], [22]–[24], [27], [29], [30] *cannot* adaptively exploit diverse optimization techniques in O1-O4 for different sub-computations, and thus they will miss the optimization opportunities brought by combining techniques from more than one category, leading to sub-optimal query latency. The research challenges include: **Challenge 1. Difficulties in unifying all co-optimization techniques in one system.** Each category of co-optimization techniques relies on a different data/computational abstraction, cost model, and transformation logic. Relational algebra optimizations (O1) rely on classical query planning rules and cost models based on the statistics and selectivity of opaque UDFs. Factorizing an ML function across joins (O2) requires analyzing the internal structure of the UDFs and the data schema together. Tensor-relational rewrites (O3) not only require analyzing the internal logic of UDFs but also converting tensor data into collections of tensor blocks for relational processing. Data-model and cross-library optimization (O4) demand reasoning about both data layout and model internals. Existing intermediate representations and systems typically support only limited combinations of these abstractions and lack mechanisms to express all required cross-level optimizations in a coherent, cost-aware way. **Challenge 2. Massive Search Space.** Our three-level Intermediate Representations (IR) has enabled full co-optimization opportunities, which enlarged the search space. Supposing there are n atomic ML functions, each function has two options, converting to relational operators or not, which leads to 2^n optimization plans in the O3 category. Then, applying O3 techniques brings new relational operators, which will expand O1’s search space. In O4, depending on how n consecutive atomic ML functions should be fused, we will have 2^{n-1} plans. If an atomic ML function is not fused with other functions, it may bring new optimization opportunities in O2 and O3. It is essential to explore IR-optimizer co-design to effectively manage this expanded search space.

To address these challenges, we present CACTUSDB, a novel UDF-centric system with:

- (1) **A Three-Level IR (Sec. III).** To address the first challenge, we advocate a three-level IR on top of the approach combining relational and linear algebra, to capture computation at different layers. The top level consists of relational operators nested with opaque user-defined expressions. At the middle level, each opaque expression at the top level is lowered to analyzable tree structures, in which each node is either an expression operator or an opaque expression. At the bottom level, each opaque expression from the middle-level IR is (optionally) described as a computational graph of ML or UDF functions. We leverage the underlying Velox system for query/expression parsing, vectorization, and physical optimization.
- (2) **A Novel Reusable MCTS Optimizer (Sec. IV).** Today’s cost-based query optimizers increasingly adopt the MCTS for efficiently exploring large search spaces, balancing exploitation and exploration via the Upper Confidence Bound (UCB) mechanism [31], and integrating with diverse models. However,

existing MCTS-based query optimizers [32]–[35] need to build a search tree from scratch for each query, as each has distinct states (query plans) and action spaces (rewrite rules), making reuse of MCTS tree challenging. We address this by combining query embeddings with MCTS to form a universal state space. Our embedding design aligns with the three-level IR, while the action space consists of co-optimization rules (O1-O4).

(3) **Implementation and A Comprehensive Evaluation (Sec. V).** We implemented CACTUSDB¹ on top of Meta’s open-sourced Velox database engine [36]. We further evaluated our system on several benchmarks, including (1) realistic recommendation queries on MovieLens dataset [37], [38] (2) TPCx-AI workloads [39], and (3) analytics workloads on three real-world datasets - Credit Card [40], Expedia [41], and Flights [41]. For the MovieLens and TPCx-AI workloads, we designed ten representative query templates for each by extracting and generalizing patterns from the most popular Kaggle notebooks. We then sampled 2,000 queries based on these templates for evaluation. We observed up to $441\times$ speedup using CACTUSDB compared to baseline systems.

II. BACKGROUND

A. SQL-ML Co-Optimization

First, without loss of generality, we assume that pure relational algebra optimization, such as join reorder and the push-down of simple predicates and projections that do not involve ML, is applied before performing co-optimization, and typical database physical optimization, e.g., join algorithm selection, is applied after co-optimization.

Then, we categorize (non-approximate) SQL-ML co-optimization techniques into four main classes, O1–O4, each defined by its distinct data abstraction (particularly how model parameters are represented) and computation abstraction.

O1. Relational Algebra Optimization [5], [7], [17], [42]. O1 performs optimization at the relational algebra level. The computation abstraction consists of relational operators such as filter, project, join, crossJoin, and aggregate. AI/ML inference is encapsulated inside user-defined expressions and is treated as an opaque function. The data abstraction consists solely of relations; model parameters (e.g., weight tensors) are not visible to the optimizer. Even when AI/ML operators are opaque, the optimizer can reduce the *number* and *placement* of AI/ML invocations by rewriting the relational query plan. Example co-optimization techniques in O1 are described in R1-1 to R1-4, as follows:

R1-1 filter Reorder. Reordering filters customized by user-defined expressions, as illustrated in Fig. 1’s R1-1, will not affect query equivalence. Reordering the filters based on their selectivities will reduce intermediate data size.

R1-2 filter Pushdown. Pushes a filter down through a join to avoid redundant evaluation on the join’s high-cardinality output that contains repeated tuples from the underlying tables, as shown in Fig. 1’s R1-2.

R1-3 project Pushdown. Similarly, project operations with user-defined expressions (including ML inference) can be pushed down through joins to avoid redundant or unnecessary computation, as shown in Fig. 1’s R1-3.

¹Open source at: <https://anonymous.4open.science/r/CactusDB>

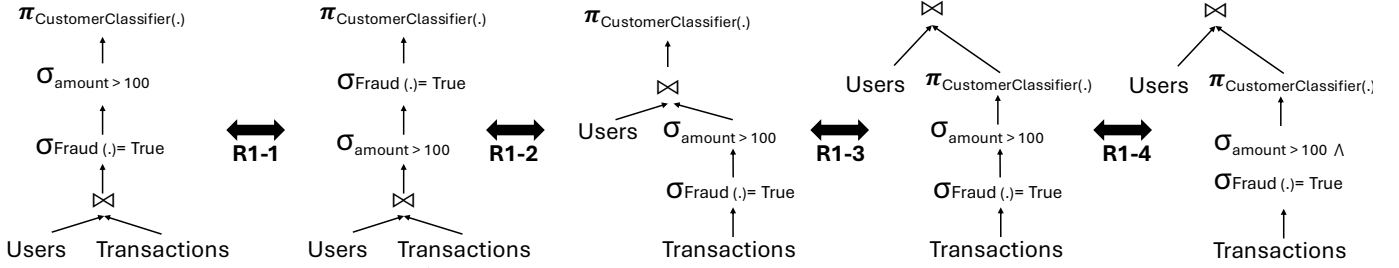


Fig. 1: Examples for R1-1 to R1-4 in the O1 category

R1-4 Operator Merge/Split. Two filters or projects over the same source or intermediate relation could be merged into one filter or project, as shown in Fig. 1's R1-4. Splitting is the reverse process of merging.

O2. Factorized Inference (derived from factorized ML [20]–[22]). The computation abstraction consists of relational, linear algebra, and ML operators, including a limited set of factorizable ML operators, such as the multiplication of a vector and a weight matrix. The data abstraction exposes model parameters as factorizable objects to the optimizer. Example co-optimization techniques include:

R2-1 Linear Algebra Operator Factorization [20]–[22]. Let $S(Y, \mathbf{X}_S, FK)$ and $R(RID, \mathbf{X}_R)$ be two relations with feature vectors $\mathbf{X}_S$ and $\mathbf{X}_R$. Joining on $S.FK = R.RID$ produces $T(Y, \mathbf{X})$, where $\mathbf{X} = [\mathbf{X}_S, \mathbf{X}_R]$. Consider a linear inference operation that computes $w^\top x^\top$ for each $x \in T.X$. Since the join replicates feature vectors from S and R across many output tuples, naively evaluating $w^\top x^\top$ on T incurs redundant computation. By partitioning the weight matrix as $w = [w_S, w_R]$, the computation can be rewritten as $w_S^\top x_S^\top + w_R^\top x_R^\top$. This factorization enables pushing $w_S^\top x_S^\top$ to S and $w_R^\top x_R^\top$ to R prior to the join, and combining the results by addition after the join, thereby eliminating repeated matrix multiplications. An example is shown in Fig. 2.

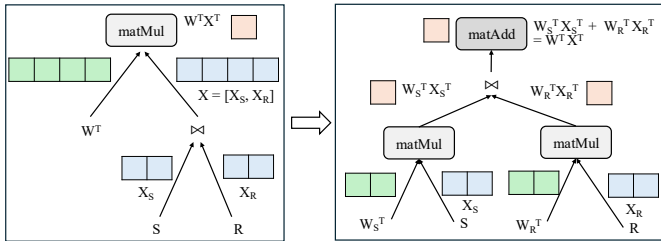


Fig. 2: Example Factorization of Matrix Multiplication (R2-1)

O3. Tensor-Relational Transformation. The computation abstraction: all AI/ML models are transformed into a limited set of tensor-relational operators [23], such as aggregate, crossJoin, filter, project, rekey, tile, concat, all processing tensor relations. In the data abstraction, the model parameters must be transformed into tensor relations. Example co-optimization techniques include:

R3-1 Linear algebra operators transformed to relational operators [23], [43]–[45]. Consider computing $Y = xW^\top$ for N input feature vectors, where x represents a row from the input table (i.e., a feature vector), and

the weight matrix W is too large to fit in memory. R3-1 stores W as a *tensor relation* of column blocks, $\tilde{W}(colId, wTile)$, and rewrites inference into a simple relational pipeline: `crossJoin`→`project`, producing block outputs $B(rowId, colId, yBlock)$. At runtime, the DB scans $\tilde{W}$ one tile at a time. For each $wTile$ loaded into the buffer pool, it computes the corresponding output block $yBlock := x \cdot wTile^\top$ for all rows, emits B , and then loads the next block. Finally, it assembles each output vector by concatenating $yBlock$ blocks per $rowId$. The process is illustrated in Fig. 3. As described in Tensor Relational Algebra (TRA) [23], other linear algebra computations can also be represented in relational algebra.

R3-2 Decision Forest to relational operators [12]. A decision forest model, like XGBoost or lightGBM, is stored as $DF(\underline{treeId} : INT, trees : DecisionTreeObject)$. Applying the model DF to each $x \in T.X$, could be transformed into relational processing by first performing a `crossJoin` of T and DF . Then a `project` is applied to each joined pair of $(x \in T.X, t \in DF.trees)$ to run $t.predict(x)$. The prediction results are aggregated to obtain the final prediction, e.g., via majority voting or a sigmoid function.

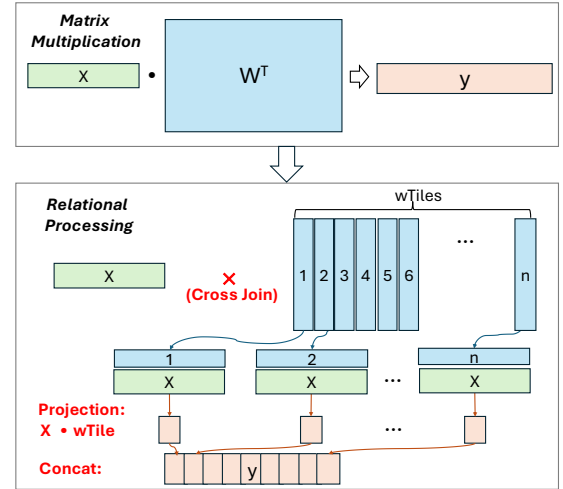


Fig. 3: Example Matrix Multiplication to Relational Processing (R3-1)

The O3 transformations scale to AI/ML models that involve a large volume of parameters, by blocking the parameters so that each time only a few blocks need to be loaded into the memory (via buffer pool management).

O4. Data-Model Cross Optimization [25], [27], [46], [47]. O4 removes the abstraction boundary between data and models. The computation abstraction includes relational operators, linear algebra operators, and AI/ML operators. The data abstraction includes both relations and tensor objects (e.g., model parameters and intermediate activations). By exposing AI/ML operators and tensor data, the optimizer can jointly

optimize data access, operator structure, and AI/ML execution. Example optimization techniques in O4 include:

R4-1 Operator Fusion and Split. Multiple consecutive AI/ML operators can be fused into a monolithic operator. For example, if we detect a sequence of UDFs that calls a matrix multiplication (`matMul`), which is followed by a matrix addition (`matAdd`), and a sigmoid operation, we may replace them with a single `DenseLayer` UDF from the `torch` library. In contrast, we could split a monolithic UDF that fuses multiple AI/ML operators into smaller ones, allowing optimization techniques in O2 and O3 to be selectively applied to sub-computations.

R4-2 AI/ML Library Replacement. An AI/ML operator’s implementation can be replaced by different libraries. For example, for `matMul`, there are different implementations for CPU, GPU, sparse matrices, and dense matrices. In addition, a `conv2D` operator can be converted into `matMul` through spatial rewriting of the input images and the filters [48]–[50], as illustrated in Fig. 4. The detailed process is explained in several references [48]–[50]. Other techniques in this category include data-induced model optimization [27], inference batch size optimization [51], etc.

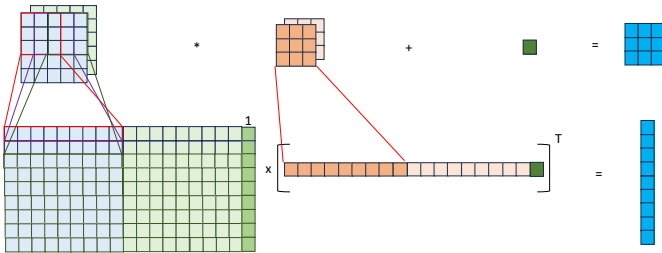


Fig. 4: Example conversion of `conv2D` (*) to `matMul` (R4-2)

In this paper, we advocate for supporting all categories of optimization techniques within a unified database system. This design enables sub-computations to leverage different data abstractions and co-optimization strategies, enhancing flexibility. Moreover, combining techniques across categories can unlock new optimization opportunities. For example, if by applying R4-1, a `DenseLayer` UDF is converted into smaller UDFs, including `matMul`, `matAdd`, and `sigmoid`, it will bring new optimization opportunities by applying R3-1 and/or R2-1 to transform the `matMul`.

B. Related Works

In-DB AI/ML Systems. To the best of our knowledge, no existing system fully supports all categories of SQL-ML co-optimization techniques (O1–O4). UDF-centric systems such as `EvaDB` [6] and `PostgresML` [7] support most O1 techniques by allowing inference logic to be encapsulated in user-defined functions. `MADLib` [17] and `PySpark` [52] provide partial support for O1, but handle arbitrary UDFs poorly. For instance, `MADLib` cannot push down table-level UDFs like `madlib.mlp_classification`, and `PySpark` only pushes down `MLlib` functions and not arbitrary UDFs. These systems cannot analyze or transform UDF internals, making them unsuitable for deeper co-optimization as required by O2–O4. Existing factorized ML systems [19]–[22], [53], [54] focus on ML training processes, relying on predefined factorizable AI/ML models or linear algebra (LA) operators. Although related to O2-style optimization, they lack dynamic decomposition of arbitrary inference pipelines and integration with O3/O4 type optimizations. `IMBridge` [51], [55] optimizes query

performance through automated ML inference context caching and batch inference. Nonetheless, it lacks support for O2 and O3, which are essential for comprehensive end-to-end performance optimization. Other systems like `SimSQL` [24], [43], `SystemML` [44], and `SystemDS` [56] embed ML workloads in relational algebra atop big data processing platforms like `Hadoop` [57] or `Spark` [58]. `MASQ` [30], [59] rewrites limited inference operators (e.g., one-hot encoding, linear models, decision trees) into SQL. None explore partial translation of LA operators into relational form for O2/O4 co-optimization.

Intermediate Representations (IRs). Several unified IRs have been proposed for SQL-ML co-optimization, though most cover only subsets of O1–O4. `Raven` [27] combines relational, LA, and ML operators, supporting O1 and O4 and partially O3 by converting full ML pipelines into relational form, reducing flexibility. `SDQL` [47] uses a semi-ring IR covering O1, O2, and O4, but cannot transform LA into relational operators (lacking O3). `Weld` [25] unifies computation across libraries through vector operators (e.g., `vecbuilder`, `vecmerger`), enabling fusion and data-movement optimization, yet most O1–O3 techniques must be implemented outside of the IR [26]. `Lara` [46] uses monad comprehensions for UDFs, enabling O1/O4 optimizations (push-down, layout tuning) but not O2/O3. `LingoDB` [60], [61] leverages MLIR for combining relational and non-relational workloads (e.g., *k*-means, PageRank) [62]. While flexible, it does not target SQL-ML co-optimization techniques in O2–O4. `Tensor Relational Algebra` [23] primarily addresses O3-style integration of tensors with relational queries, without broader support for O1, O2, or O4.

Inference Query Optimization. `Raven` [27] applies logical optimizations in a fixed order using rule-based, classification, and regression strategies to choose among `ML2SQL`, `ML2DNN`, or no optimization. Most other in-database AI/ML systems rely on rules and heuristics for inference query optimization. `Weld` [25] and `Lara` [46] use rule-based plan selection, while `SDQL` [48] applies rewrite rules until reaching a fixed point.

Database query optimization has evolved into a multifaceted research domain, encompassing rule-based, cost-based, and learning-based approaches. **Rule-based optimizers (RBOs)** apply predefined transformations (e.g., `Spark Catalyst` [63], [64]), but fixed ordering can yield suboptimal plans. **Cost-Based Optimizer (CBO)** [65]–[67] estimates the cost of query execution using statistical models and employs techniques such as dynamic programming, genetic algorithms, and Monte Carlo Tree Search (MCTS) [32], [35]. **Learned optimizers (LBOs)** [5], [32], [68], [69] use ML for cost or cardinality estimation but struggle with ad-hoc AI/ML queries unseen during training.

Our Work. To close the gaps, we proposed `CACTUSDB`, a novel in-database AI/ML system focusing on inference queries. Like `Raven` [27], and different from `Weld` [25], `Lara` [46], `LingoDB` [61], and `SDQL` [47], `CACTUSDB` allows relational, linear-algebra, and ML operators to co-exist for expressive and flexible query processing. But our system is distinct from `Raven` in the following aspects: (1) Unlike `Raven`’s ONNX-based IR, `CACTUSDB` encodes AI/ML semantics using

native database expressions in UDF-centric systems, forming a three-level IR. This yields tighter integration with existing optimizers, improved partial evaluation, and seamless adoption in SQL/UDF systems. (2) Instead of treating ML pipelines as monolithic graphs, CACTUSDB decomposes inference pipelines into fine-grained sub-computations, enabling selective optimizations such as factorization, push-down, and tensor-relational conversion. (3) While Raven explores a small rule-based search space (ML2SQL, ML2DNN, or none), CACTUSDB introduces a new optimizer combining query embeddings with MCTS to efficiently search the exponentially large space of co-optimization strategies.

III. THREE-LEVEL IR

In this section, we introduce our data and computation abstractions and describe our three-level IR design and usage.

A. Flexible Data Abstraction

We provide flexible abstractions for features and model variables:

Feature Vectors are extracted from relations and stored alongside other attributes, denoted as $F(A_1 : Type_1, \dots, A_l : Type_l, V : \text{vec} \in \mathbb{R}^d)$, where A_i is an attribute of type $Type_i$, and V represents the stored feature vectors. If V concatenates m vectors $V_1, \dots, V_m$, where $V_1, \dots, V_m$ are from datasets $D_1, \dots, D_m$, respectively, we define $V \equiv [V_1, \dots, V_m]$.

Model Parameters are variables of ML functions, which are created when models are loaded into CACTUSDB. A variable can be a tensor (deep learning), a tree collection (decision forest), or other types of parameters. A variable could be stored as a relation. For example, a d -dimensional weight tensor W_i can be stored as a relation $P_i(dim_1 : INT, \dots, dim_d : INT, block : \text{vec} \in \mathbb{R}^{k_1 \times \dots \times k_d})$, where each tuple represents a d -dimensional tensor block indexed by $dim_1, \dots, dim_d$, with the block data of shape $k_1 \times \dots \times k_d$ flattened as an array in the column *block*. Similarly, a decision forest can be stored as a relation $DF(TreeId : INT, TreeObj : TreeType)$, where each tuple represents a tree object indexed by *TreeId*. To enable efficient conversion, CACTUSDB selectively materializes model variables as relations during loading if their size exceeds a threshold (typically half the available memory).

B. Three-Level Computation Abstraction

CACTUSDB supports three types of operators:

Relational Operators. CACTUSDB supports most relational operators such as *filter*, *project*, *join*, *crossJoin*, *union*, *aggregate*, and *expand* (i.e., similar to *flatMap*). Each such operator processes a relation, such as F, P_i, DF .

Expression. Most relational operators should be customized by expressions, which are in the form of analyzable tree structures. A tree node can either be an opaque expression or an expression operator, such as arithmetic (+, -, *, /), comparison (>, <, =, ≠), bit (e.g., ∧, ∨), relation (e.g., AND/OR), conditional (e.g., IF/THEN/ELSE and SWITCH), and function call (e.g., CALLFUNC) operators. Each expression operates on one or multiple columns, e.g., $A_i, V, \dim_j, \text{block}$.

ML Functions. In CACTUSDB, we have implemented a set of ML functions. An ML function is a specialized Velox UDF. Each ML function class has an *apply* method, which takes a batch of vectors or tensor blocks as input to facilitate the use of SIMD instructions and reduce virtual function call overheads. Although this work focuses on CPU-based database systems, GPU acceleration can also be enabled for many ML functions by passing a flag to the *apply* method. The supported functions include: (1) **Atomic ML Functions** such as *matMul*, *matAdd*, *conv*, *softmax*, *argmax*, *pooling*, *relu*, *batchNormalization*, *cosSim* (cosine similarity), *concat*, *dropout*, *embed* (embedding lookup), *decision Tree*, *standard/min/maxScalarization*, *seqEncode* (i.e., encoding a sequence of tokens into an embedding vector), and *binarization*; (2) **High-level ML Functions** such as feed-forward neural network model (*ffnn*), convolutional neural network (*cnn*), decision forest model (*xgboost/lightgbm*), Huggingface inference endpoints (*huggingface*), and ChatGPT APIs (*llm*). These functions take one or more vectors/tensor objects as input, e.g., $W_i \in W, \text{vec} \in V, \text{vec} \in \text{block}$ as described in Sec. III-A.

Importantly, a user can register customized atomic and high-level functions in addition to these built-in ML functions.

C. IR Overview

To maximize co-optimization opportunities by enabling each sub-computation to select its optimal computation and data abstraction, we propose a novel three-level IR:

Top-level IR: Nested Relational Algebra. In the top-level IR, every node is a relational operator, which is customized by an expression, each edge represents a data flow from the source node to the destination node. Each expression is opaque at this level.

Middle-level IR: Expression Tree. An opaque expression from the top-level IR is lowered to the middle-level IR as an expression tree, where each node is either an expression operator (as described in Sec. III-B) or an opaque sub-expression.

Bottom-level IR: ML Computation Graph. An opaque sub-expression in the middle-level IR is lowered to the bottom-level IR as a computation graph that is understandable to the query optimizer. In the bottom-level IR, each node represents an atomic ML function, as described in Sec. III-B, while each edge represents a tensor output from the source node and input to the destination node. From the bottom-level IR, each atomic ML function’s details, such as input tensor shapes, the shapes of model weight matrices, etc., can be retrieved by the query optimizer from the ML function instance through pre-defined interfaces. At this level, multiple atomic ML functions can be fused into one high-level ML function or vice versa. In addition, any ML function may be replaced by an equivalent ML function with different implementation.

Our IR design allows a query optimizer to rewrite more than one level of IRs at the same time to support any co-optimization techniques from O1-O4 in arbitrary ordering for

any sub-computation, maximizing co-optimization space, as shown in the next section.

D. Workflow: from a Query to Our IR

As illustrated in Fig. 5, at Step 1, a user loads a two-tower recommendation model, which is a popular type of recommendation models [18], [70]. In our example, the model consists of a user tower and a movie tower. The two towers encode user features and movie features into vectors, respectively. The cosine similarity between the user vector and the movie vector is then computed to decide whether the movie should be recommended to the user. During the model loading process, all ML functions required by the model are customized, registered, and used to compose a computational graph using the customized ML functions. At Step 2, the computational graph is registered as the `twoTowerModel` expression. At Step 3, a query is issued that applies the `twoTowerModel` expression to compute a recommendation score for each pair of preprocessed user and movie features. At Step 4, the query is translated into a three-level IR, where the top `Project3` node in the top-level IR points to a middle-level IR that only contains the opaque `twoTowerModel` expression, which links to the detailed bottom-level IR registered at Step 3.

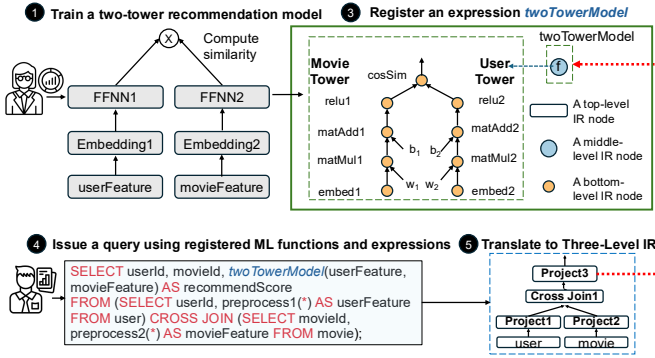


Fig. 5: Develop and Translate an Inference Query to Our IR.

E. Examples of Using Our IR

Fig. 6 presents three transformations of the 3-level IR from our running example in Fig. 5.

As shown in Fig. 6-1, the optimizer first splits the `twoTowerModel` expression into three parts: the user tower, the movie tower, and an operator that computes their cosine similarity (See R4-1 in Sec. II-A). In Fig. 6-2, the node `Project3` in the top-level IR is split into `Project3'` corresponding to cosine similarity, `Project4` corresponding to movie tower, and `Project5` corresponding to user tower (See R1-4 in Sec. II-A). Then, in Fig. 6-3, `Project4` and `Project5` are pushed down to avoid redundant computations (See R1-3 in Sec. II-A).

To illustrate more transformation rules, we provide a variation of the running example in Fig. 5 by changing the two tower recommendation model into DNN-based recommendation model that has two dense layers. In addition, we also added a filter on the movie features, while the filter uses another two-layer DNN model to predict and decide whether the input tuple

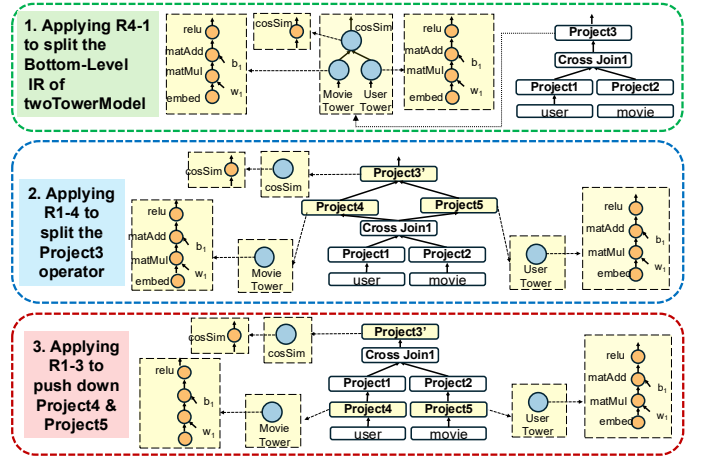


Fig. 6: Example Transformations of our three-level IR.

should be kept or not. Fig. 7 uses this variation to illustrate the flexibility and expressiveness of our three-level IR. In Fig. 7(a), a query is issued to extract, cross join, and concatenate movie features and user features, and then it applies a DNN model (the `recommendDNN` function) to each concatenated [user, movie] feature vector to calculate the corresponding recommendation score. The results are further filtered using another DNN model (the `popularMovieClassifier` function). A default query plan represented in our three-level IR is shown in Fig. 7(b), where the `Project3` is customized by the `recommendDNN` function, and `Filter1` is customized by the `popularMovieClassifier` function. The details of these functions are described by the middle-level IR consisting of expression operators (i.e., blue nodes). Each opaque expression can be further lowered to bottom-level IR consisting of ML functions (i.e., orange nodes). In Fig. 7(c)-(f), we show how our IR supports different co-optimization techniques from O1-O4.

First, pushing down a filter that is customized by a ML function (R1-2 in O1) can be easily performed at the top-level IR, as shown in Fig. 7(c), where `Filter1` is push down to the `movie` table without changes to other levels of IRs.

Next, as shown in Fig. 7(d), two optimization techniques are applied. First, it applies R2-1, which factorizes the `matMul3` from the bottom-level IR of the `denseLayer3` expression into `matMul3-1` that multiplies the co-factorized weight matrix w_{3-1} with the preprocessed movie features, and `matMul3-2` that performs a similar logic to preprocessed user features. As a result, the `Project3` is factorized into `Project3'`, `Project4`, and `Project5`. `Project3'` corresponds to the rest of the FFNN processing (e.g., the rest of `denseLayer3` and `denseLayer4`). `Project4` and `Project5` correspond to the factorized `matMul3-2` and `matMul3-1` respectively. They are atomic ML functions and thus they are not lowered to bottom-level IRs. Second, R1-3 is applied to push down both `Project4` and `Project5` through the `CrossJoin1` to avoid redundant computations.

Then, in Fig. 7(e), R3-1 is applied to convert the `matMul3-1` and `matMul3-2` into relational operators. In

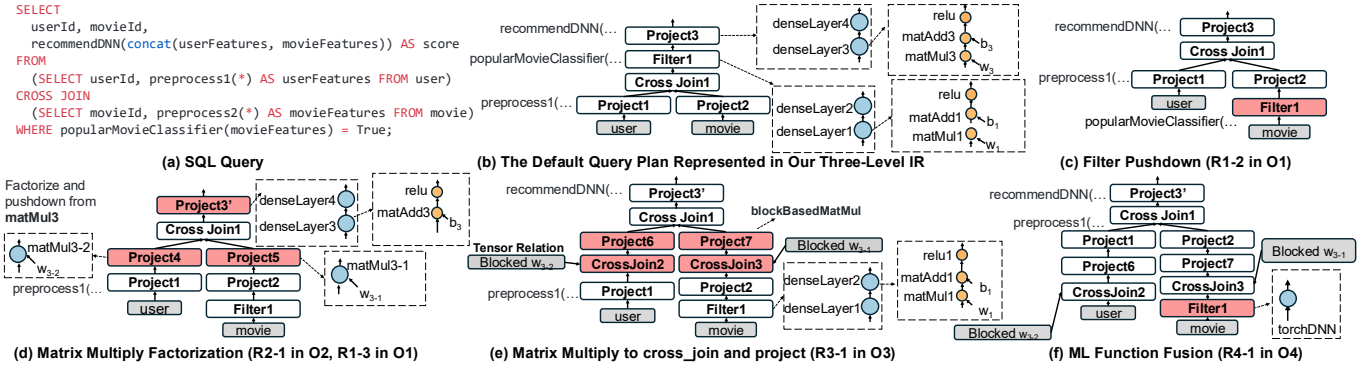


Fig. 7: Examples of our three-level IR. Details are described in Sec. III-E.

this case, the weight matrices w_{3-1} and w_{3-2} are both vertically partitioned into tiles, e.g., $P3_1(colId : INT, tile : t \in \mathbb{R}^{d1 \times m})$ and $P3_2(colId : INT, tile : t \in \mathbb{R}^{d2 \times m})$, where $d1$ and $d2$ are factorized feature dimension sizes, and m represents the first hidden layer size of the model *recommendDNN*. Therefore, *matMul3-1* is converted into *CrossJoin3* that combines *P3_1* and the output relation of *Project2*, i.e., the preprocessed movie features in the form of $I(movieId : INT, V : x \in \mathbb{R}^{d1})$, where each tuple represents a $d1$ -dimensional movie feature vector indexed by *movieId*, followed by *Project7* that multiplies each joined pair of $t \in P3_1.tile$ and $x \in R.V$. The transformation of *matMul3-2* is similar.

Finally, as shown in Fig. 7(f), it applies R4-1 to fuse all operators in the *popularMovieClassifier* model into one high-level ML function that invokes the *torchDNN* library.

IV. QUERY OPTIMIZATION

In this section, we will describe our unique query optimization process co-designed with our three-level IR representation. The goal is to address the search space challenge via a novel reusable MCTS model based on query embedding techniques.

A. MCTS Preliminary

Recently, MCTS has become a popular tool for query optimization [32], [35]. That is because MCTS can efficiently explore a large search space, e.g., various rewrite ordering, by balancing exploration and exploitation, guaranteeing asymptotic convergence to the optimal decision once combined node selection strategies such as UCB [71]. A common abstraction of MCTS for query optimization includes the following components:

States, i.e., a node in the MCTS tree, represents a logical query plan of the SQL query to be optimized and the input datasets.

Node Utility Function. Every state is associated with a utility representing the cumulative reward of the state and indicating the probability that the state is in the optimal path.

Actions. Usually, an action, i.e., an edge in the MCTS tree, refers to a rewriting or equivalent transformation of the current plan, transitioning it to a new plan that produces equivalent results. The applicable rewriting and transformation rules defines the action space.

Runtime Search Process. The MCTS algorithm starts with the root node, the initial query plan. At each step, it checks

whether the current state has been expanded, i.e., all rewriting rules of the given state have been explored. If so, the MCTS selects an action based on UCB (Line 1 of Alg. 1), where r_i is the utility of $node_i$ (the accumulative reward of $node_i$), n_i is the number of total visits of the $node_i$, N_i is the total number of visits of the parent node of $node_i$, and c is a constant for balancing exploration ($\frac{r_i}{n_i}$) and exploitation ($\sqrt{\frac{\ln N_i}{n_i}}$).

Algorithm 1: *select(node)*

Input: *node*: the current state of the given query

- 1 $selectedNode \leftarrow \underset{node_i \in node.children}{\operatorname{argmax}} \frac{r_i}{n_i} + c \sqrt{\frac{\ln N_i}{n_i}};$
- 2 **return** *selectedNode*;

If the current node is not fully expanded, it will expand the node by randomly selecting one unexplored action (see Alg. 2).

Algorithm 2: *expand(node)*

Input: *node*: the current state of the given query

- 1 *unexplored* $\leftarrow \{a \in node.actionSpace \mid a.explored == False\};$
- 2 *selectedAction* $\leftarrow \operatorname{randomSelect}(unexplored);$
- 3 *selectedAction.explored* $\leftarrow True;$
- 4 *node'* $\leftarrow node.takeAction(selectedAction);$
- 5 *node.children* $\leftarrow node.children \cup \{node'\};$
- 6 **return** *node'*;

Then, it performs a simulation, also called a rollout, as described in Alg. 3. The rollout process randomly chooses an action until it reaches a maximum number of iterations or the terminating state (i.e., no more applicable rules). After rolling out, it will run a back-propagation process to compute rewards and update the utility of each selected node, described in Alg. 4. See supplementary material for the overall algorithm.

Algorithm 3: *rollout(node)*

Input: *node*: the node where simulation starts

- 1 **while** *node.isTerminateState()* $== False$ **do**
- 2 $selectedAction \leftarrow \operatorname{RandomSelect}(node.actionSpace);$
- 3 $node \leftarrow node.takeAction(selectedAction);$
- 4 **return** *node*;

Shortcomings of the naive MCTS optimization. As a runtime query planner, MCTS constructs a search tree from scratch for every query, making optimization inefficient. That

Algorithm 4: *backpropagate(node, reward)*

Input: *node*: the node where simulation start; *reward*: the reward value obtained from reward function

```
1 while node != NULL do
2    $n_i \leftarrow n_i + 1$ ;
3    $r_i \leftarrow r_i + \text{reward}$ ;
4    $\text{node} \leftarrow \text{node.parent}$ ;
5 return;
```

is because different queries have unique states and action spaces, preventing tree reuse even among similar queries. This is problematic since real-world inference queries often share common sub-queries. For example, recommendation systems may query multiple models with similar architectures but trained on different datasets with varying hyperparameters for the same user profile [72]. Pretrained models, such as BERT, are also reused across different datasets using a similar query template [73], [74].

Therefore, the naive MCTS model cannot exploit such similarity to reduce query optimization latency.

B. Reusable MCTS based on Query Embedding

Recent Query2Vec techniques encode SQL queries into fixed-length vectors for downstream tasks such as query optimization (e.g., BAO [69], NEO [75]), cost estimation (e.g., E2E-Cost [76], RTOS [77]), and join/index selection (e.g., ReJOIN [78], AI Meets AI [79]). More recently, QueryFormer [80] leverages transformers for general-purpose embeddings. We build on this line of work by encoding logical query plans into embedding vectors to enable MCTS to generalize across queries and improve optimization efficiency. However, existing methods do not support our three-level IR and cannot be directly applied.

Challenge-1. *Existing query2vec models did not consider queries that nest with AI/ML inferences, and they cannot capture expressions and ML functions at the middle and bottom-level IRs.*

Challenge-2. *While the query embedding can be used to unify the state space for different queries, the action space remains query-specific. How to integrate Query2Vec with MCTS to navigate the full co-optimization space is still unclear.*

To address the challenges, we offer a novel reusable MCTS framework co-designed with our three-level IR. We choose to build our work on top of QueryFormer [80], because it represents the state-of-the-art query embedding technique that is capable of capturing the structural information of the query plan, and it is used in various tasks, including query optimization and cost estimation [81].

1) *Embedding the Multi-level IR:* QueryFormer [80], like other query embedding methods, is designed for tree-based query plans rather than query plans represented in our three-level IR. As illustrated in the Step 1 of Fig. 8, which follows the running examples in Fig. 5 and Fig. 6, it encodes each query node in a query tree to a vector. This is done by encoding each node’s attributes such as *operator type*, *join type*, *input table*, *filter predicate*, *histograms of the related columns*, and

bitmap of samples into embedding vectors E_o , E_j , E_t , E_p , E_h , and E_s respectively. These vectors are concatenated to form a fixed-length node vector. Each node vector is further added with a height encoding vector that specifies the node’s position in the query graph. The query graph is then abstracted as a sequence of node vectors following the in-order tree traversal. The sequence is sent to a transformer model, which outputs the final embedding vector for the entire query tree.

To address **Challenge-1**, we adopt a recursive encoding strategy that captures the rich hierarchical structure of the three-level IR. At the bottom level, each opaque expression is encoded into an embedding vector E_{expr} . These are then composed into middle-level IR embeddings: E_p for a *filter* predicate and E_h for the histogram of the corresponding *filter* column by trivially extending QueryFormer’s methods of encoding E_p and E_h . Finally, the top-level IR—comprising these embedded relational operators—is processed by the QueryFormer model to generate the final query embedding.

Embedding of Bottom-Level IR. We employ a QueryFormer-like, transformer-based Model2Vec to embed a bottom-level IR into a vector representation in the following approach: (1) For each opaque expression at a middle-level IR, a **breadth-first traversal** is applied to its corresponding bottom-level IR (if present), to encode each node (e.g., an ML function) into a feature vector representation by concatenating the following components:

- E_{mlType} : Encodes the ML function type, preserving function diversity. It is generated through an embedding layer, where the parameters are learned during the training process.
- $E_{mlFlops}$: Captures the computational cost of the node as the number of floating point operations (FLOPs), a scalar value.
- E_{mlDims} : Records the shape of the tensor dimensions.

(2) The resulting bottom-level IR graph, where each node is represented as a feature vector, is then passed to the Model2Vec to generate the final embedding vector, E_{expr} , for the expression.

Embedding of an Entire Query. Similar to QueryFormer, for each relational operator in the top-level IR, we first extract feature vectors for E_o , E_j , E_t , E_p , E_h , and E_s . Here, if the operator’s middle-level IR contains an opaque expression that maps to a bottom-level IR encoded as E_{expr} , E_i is constructed similarly to QueryFormer [80] except that it sees the bottom-level IR’s output as an output column encoded by E_{expr} . Then, each feature vector E_i is scaled to a fixed size through a linear layer. The resulting vectors are then concatenated to construct the top-level node embedding vector:

$$V = \parallel \text{LinearLayer}_i(E_i), i \in \{o, j, t, p, h, s, expr\} \quad (1)$$

where $\parallel$ is the concatenation. We then map the query plan into a sequence of m vectors ($V_1, \dots, V_m$) corresponding to m top-level IR nodes following in-order tree traversal of the top-level IR, which is sent to a QueryFormer-like Query2Vec model [80] to output a final embedding vector for the query plan represented in our three-level IR, as illustrated in Step 2 of Fig. 8. Additionally, this design enables our cost model

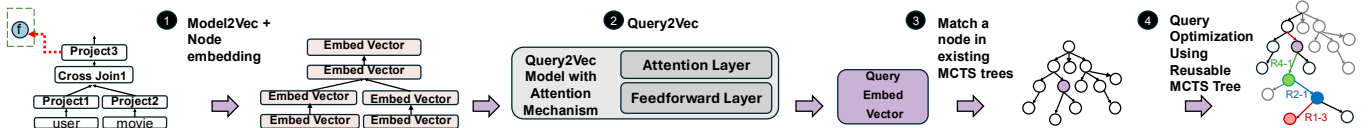


Fig. 8: CACTUSDB's Query Optimization Workflow.

to handle selectivity using two complementary mechanisms. First, for native SQL filters, selectivity features are explicitly encoded via the histogram and filter predicate embeddings. Second, for AI/ML filters, selectivity is learned implicitly by combining the model embedding with the table and column embeddings. During the training process, the model learns the latent relationships between the model and the model's input data distribution, capturing the predicate's selectivity.

While the architectures of our Model2Vec and Query2Vec models are similar to QueryFormer [80], we redesign the loss function for two tasks: (1) query embedding for matching a query to an MCTS state; and (2) execution latency prediction for MCTS reward computation. We train these two tasks separately.

Task 1. Query Embedding for MCTS State Matching. We combine contrastive learning with Weisfeiler-Lehman (WL) kernel, a widely used and effective method for capturing structural similarity of graphs [82], to map semantically similar expressions and query plans closer in the Model2Vec and Query2Vec embedding spaces while pushing dissimilar ones apart [83], [84]. The corresponding contrastive loss is defined in Eq. 2, where N is the number of training samples, $\mathbf{z}_i$ denotes the embedding vector of the anchor sample, $\mathbf{z}_i^+$ and $\mathbf{z}_i^-$ are its positive and negative examples, $\text{sim}(\mathbf{z}_1, \mathbf{z}_2)$ denotes cosine similarity $\frac{\mathbf{z}_1^\top \mathbf{z}_2}{\|\mathbf{z}_1\| \cdot \|\mathbf{z}_2\|}$, and τ is a temperature hyperparameter.

$$\text{Loss}_{\text{contrastive}} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i \quad (2)$$

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau)}{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^-)/\tau) + \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau)} \quad (3)$$

To train our Model2Vec and Query2Vec models via contrastive learning, we construct positive and negative pairs using structural similarity quantified by the WL subtree kernel [85]. In each WL iteration, node labels are updated by hashing the current label with the multiset of neighbor labels, capturing rich local structures. Each graph is represented by a feature vector of normalized label frequencies, and similarity is computed via cosine similarity. In Model2Vec, node labels are initialized by ML function type and FLOPs in the bottom-level IR. Functions of the same type share the same label only if their FLOPs differ within a threshold. In Query2Vec, the initial label for each relational operator encodes operator type, input table, columns, and the WL features of associated expressions. Although WL features define contrastive pairs, they cannot replace embedding models at runtime, as unseen queries may yield varying label counts and inconsistent feature lengths.

Task 2. Latency Prediction for Cost Estimation. We leverage the Query2Vec embeddings to predict the query execution latency, which is incorporated into MCTS action configuration

and reward function. This prediction is implemented via a four-layer FFNN that takes embedding produced by Query2Vec as input and predicts the estimated latency. The loss function defined in Eq. 4, is based on mean squared error (MSE), where $\hat{y}_i$ and y_i denote the predicted latency and ground-truth latency of the i -th sample, respectively.

$$\text{Loss}_{\text{latency}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (4)$$

Among possible training strategies, we adopt a two-model design. Query2Vec is first trained via contrastive learning for embedding generation, and a separate copy is retrained with an FFNN for latency prediction. We chose this decoupled approach because joint training of both objectives (contrastive loss for state representation and MSE loss for cost estimation) resulted in gradient interference, degrading the cost estimation accuracy. By isolating the tasks, we achieved better query plan cost estimation, resulting in an improvement in end-to-end execution latency compared to the single-model strategy as discussed in Sec. V-E.

2) *Reusable MCTS with Configurable Actions:* To address the **Challenge-2**, we proposed a unique MCTS abstraction:

Embedding-based States. We use our recursive embedding strategy described in the previous sections to convert each logical query plan into a 393-dimensional embedding vector to represent a state. This dimensionality is determined by concatenating the feature vectors defined in Eq. 1. Following the original QueryFormer [80] architecture, we set the embedding dimension to 64. The final dimension is the sum of five components plus filter predicate embedding, which combines the filter column embedding (64), operator embedding (8), and a normalized scalar value (1).

Configurable Actions. Each action corresponds to a high-level co-optimization rule (Sec. II), making the action space universal across queries. When a rule is selected via UCB, we configure the rule to decide how to apply it to the query. For example, if R3-1 is selected, we must choose a `matMul` function to convert to relational operators. We first use heuristics, if available, to narrow the candidates, e.g., we select the `matMul` functions involving the top- k largest tensors. We then identify the optimal candidate that maximizes cost reduction under R3-1 using our embedding-based latency predictor as the cost estimator. The configured rule is then applied to transition the current state to the next state, transforming the query plan. This approach allows for a universal action space, addressing Challenge-2.

Runtime Search. A subtree of an MCTS tree may be shared by query plans with similar embedding vectors. As illustrated in Alg. 5, in the online planning process, a query's default plan will be converted into an embedding vector using our Query2Vec model, denoted as M_{Q2V} . As illustrated in Step 3

Algorithm 5: Search with Reusable MCTS

Input: *query*: a query applied with simple predicate push-down and join reordering; M_{Q2V} : a model embedding *query* to a vector; $B_iteration$: the maximum number of iterations; *mctsTreeSet*: A set of existing MCTS trees; *nodeIndex*: An index of embedding vectors of existing MCTS tree nodes.

```
/* Convert the query into an embedding vector */
1 queryEmbed  $\leftarrow M_{Q2V}(\text{query})$ ;
/* Find nearest neighbor in the MCTS trees */
2 if nodeIndex.count(queryEmbed) then
    /* Start training from the searched node */
    3 root  $\leftarrow \text{nodeIndex.getNearestNeighbor}(\text{queryEmbed})$ ;
4 else
    /* Otherwise, initialize a new MCTS tree */
    5 root  $\leftarrow \text{TreeNode}(\text{queryEmbed})$ ;
    6 mctsTreeSet.add(root);
7 for i in range(B_iteration) do
    8 node  $\leftarrow \text{root}$ ;
    9 while node.isTerminateState() == False do
        10 if node.expanded then
            11 node  $\leftarrow \text{select}(\text{node})$ ;
        12 else
            13 node  $\leftarrow \text{expand}(\text{node})$ ;  $T \leftarrow \text{rollout}(\text{node})$ ;
            14 break;
    15 reward  $\leftarrow \text{computeReward}(T)$ ;
    16 backpropagate(node, reward);
/* After search, update node indexes */
17 nodeIndex.update(node);
18 return;
```

of Fig. 8, it then performs a nearest neighbor search across the nodes in existing MCTS trees. If there is no matching state with which the cosine similarity is smaller than a threshold, a new MCTS tree needs to be constructed from scratch. Otherwise, it will resume the MCTS search process starting from the matched state, as shown in Step 4 of Fig. 8. Note that in the highlighted sequence of transformation rules that are selected by MCTS, we have R4-1, followed by R2-1, and R1-3, as explained in Sec. III-E.

Reward. Upon reaching a terminate state, T , MCTS computes the reward as the overall cost reduction, as the overall cost reduction of the plan, i.e., $\text{reward} = \text{cost}_{\text{root}} - \text{cost}_T$. This reward is then backpropagated following Alg. 4, while the remaining search process follows the general MCTS procedure described in Sec. IV-A.

3) *Theoretical Error Bound Analysis.*: We model CactusDB’s reusable MCTS-based query optimization as search in a depth- d expectimax tree [86] rooted at the initial logical plan s_0 . Let $\mathcal{S}$ denote ground states (logical plans), $\mathcal{A}$ rewrite actions, and $\mathcal{T}$ the set of ground trajectories up to depth d . The optimal action-value and value functions over trajectories are defined as

$$Q^*(t, a) = \mathcal{R}(t) + \sum_{t'} \mathcal{P}(t, a, t') V^*(t'), \quad (5)$$

$$V^*(t) = \max_{a \in \mathcal{A}} Q^*(t, a). \quad (6)$$

CactusDB applies a query embedding function $f : \mathcal{S} \rightarrow \mathcal{X}$ to aggregate ground states into abstract states, inducing a partition of trajectories into abstract histories $\mathcal{H}$. Reusable MCTS shares statistics among trajectories within each abstract

history, effectively optimizing over the resulting abstract expectimax tree. Following prior analysis of MCTS with state aggregation [86], we assume the embedding f satisfies (p, q) -consistency: for every abstract history $h \in \mathcal{H}$, there exists an action that is uniformly p -near-optimal for all ground trajectories in h , and the optimal values of any two trajectories in h differ by at most q . Under this assumption, the abstraction-induced error accumulates linearly with search depth.

Theorem 1 (Reusable MCTS error bound under (p, q) -consistency). *Let $a^\#$ be the action selected at the root by optimizing over the abstract MCTS tree. Then the suboptimality of $a^\#$ in the ground MDP is bounded by*

$$\left| \max_{a \in \mathcal{A}} Q^*(s_0, a) - Q^*(s_0, a^\#) \right| \leq 2d(p + q).$$

The full proof and a detailed discussion of the abstraction-induced error decomposition are provided in Appendix C.

We additionally derived an empirical error bound using Latin Hypercube Sampling (LHS) to systematically cover the query embedding space. Specifically, we sampled 200 queries (from queries in Sec. V-C5), and for each query, used our reusable MCTS framework to generate a transformation plan and measure its execution latency. We then performed exhaustive search to obtain the ground-truth optimal latency. Across all sampled queries, we observed a mean error of 0.0228 seconds with a standard deviation of 0.0150 seconds. The corresponding 95% confidence interval ranges from 0.0207 to 0.0249 seconds.

V. EXPERIMENTAL EVALUATION

In this section, we present our implementation and compare the end-to-end inference query latency of CACTUSDB with baseline systems, followed by an ablation study on the benefits of integrating co-optimization rules. We also evaluate the effectiveness and overheads of our query optimization strategy.

A. Implementation

We implemented our three-level IR and our reusable MCTS query optimizer using Velox [36] as a backend. We leveraged Velox’s query parser and expression parser to support the top-level and middle-level IR, while implementing all ML functions to support the bottom-level IR on our own. The action space of our query optimizer includes all co-optimization rules as discussed in Sec. II-A. Our optimizer is applied after rule-based join reordering is executed and before Velox’s physical query optimization (e.g., filter reordering and adaptive column prefetching [36]) is performed. All existing MCTS tree nodes are stored in Faiss [87], a well-known vector database, with cosine similarity indexing to accelerate nearest neighbor search.

To generate training data for Model2Vec, we defined a set of model templates—including two-tower recommendation models, Deep Learning Recommendation Model (DLRM) [88], autoencoders, singular value decomposition (SVD) [89], convolutional neural networks, FFNN, embedding models, decision forests, and others (details are provided in the *supplementary document*). From these templates, we randomly sampled

10,000 ML models of varying hyperparameters (e.g., number of neurons, number of layers, number of trees, tree depth, etc.) to construct the training data, with 80% for training and 20% for testing. The trained Model2Vec model is used for embedding the ML expressions to build the feature vectors for the Query2Vec model as described in Sec. IV-B1. The downstream model inference latency prediction task achieves an average Q-Error [80], [90] of 1.1 and a Pearson correlation of 98.02%, demonstrating its effectiveness in capturing the complexity of ML models. (Q-Error, defined as $Q(c) = \max(\frac{actual(c)}{predicted(c)}, \frac{predicted(c)}{actual(c)})$, is widely used for cost estimation [80], [90].)

To train the Query2Vec model, we generated 10,000 training queries using 14 out of 20 query templates described in Sec. V-C5. We further split them into 80% training and 20% testing sets to evaluate the model’s performance. Our two-model strategy achieved a median Q-Error of 1.19 and 94% correlation between predicted and actual latency, outperforming the one-model strategy, which achieved a median Q-Error of 1.6 and 90% correlation. It is important to note that for query optimization, preserving the relative ranking of query plans is more critical than absolute prediction accuracy. The high correlation confirms that our model effectively preserves the relative ranking of candidate query plans. This allows MCTS to reliably explore the search space.

B. Baselines

We compared the end-to-end query processing latency of CACTUSDB to the following baselines:

EvaDB (v0.3.9), which uses Postgres as backend, provides an AI-centric query optimizer focusing on result caching, query predicate reordering, and parallel query processing [6], [42].

Apache PySpark w/ UDF (v3.5.0), which allows model inferences via UDFs [91], [92], with data stored in HDFS.

MADLib (v2.1.0), which extends Postgres with ML functions [17].

SystemDS (v3.2.0), provides a declarative language for data science and ML pipelines, built on Apache Spark [44], [56].

PostgresML (v2.10.0), a Postgres extension for ML [7].

IMBridge, accelerates in-DB ML inference by caching model contexts and batching predictions efficiently.

DL-Centric, where data is stored in Postgres database (v14.15), and then data is transferred into PyTorch (v2.1.2) using ConnectorX (v0.3.2) [93] for model inferences. Its end-to-end query latency includes both in-database processing time, data transfer time, and model inference time.

We also compared the end-to-end latency and the query optimization latency of our reusable MCTS query optimizer with the following optimization algorithms, all applied after join-reordering and before Velox physical query optimization. **Un-optimized**. The query runs without co-optimization.

Arbitrary Co-optimization. It scans all co-optimization rules and applies all applicable rules [32].

Heuristics-based. (1) it will aggressively push down the filter/project operators; (2) it will aggressively fuse ML operators; (3) Tensor relational transformations are applied

only if the model size exceeds a threshold (half of the available memory size).

Vanilla MCTS. It builds an MCTS search tree following the naive MCTS search process as described in Sec. IV-A. Both our proposed MCTS optimizer and vanilla optimizer use the learned cost estimation model based on query embeddings, described in Sec. IV-B1.

Reusable. We evaluated two versions of our reusable MCTS optimizer with the embedding and cost estimation models trained using the one-model and two-model strategies respectively, as described in Sec. IV-B1 and Sec. V-A.

Evaluation Environment. Most experiments were conducted on an AWS r4.2xlarge instance (8 CPU cores, 61 GB memory). The query optimizer evaluation ran on an Ubuntu machine (48 CPU cores, 125 GB memory). GPU experiments used an AWS g4dn.4xlarge instance (16 CPU cores, one T4 GPU with 16 GB memory and 64 GB host memory).

C. Benchmarks

We used five benchmarks, reflecting realistic industrial query scenarios. These benchmarks nest more than ten diverse AI/ML model architectures with complex SQL queries. Most queries involve up to 10 ML functions, 1 – 4 join/crossJoin operators, 2 – 5 filters, 1 – 7 projection operators, and up to 4 aggregate/groupBy operators, ensuring a comprehensive evaluation of co-optimization strategies. See *supplementary document* for detailed queries.

1) *Recommendation Queries*.: The MovieLens dataset [37], [38] is widely used for evaluating DB-for-ML works [38], [44], [94]–[96]. We used MovieLens-1M dataset, which collected 1 million ratings from 6,000 users for 4,000 movies and augmented it with movie tag relevance data from MovieLens-32M dataset, which provides 140,979 unique tags and per-movie relevance vectors. We developed three realistic recommendation queries based on existing studies. Q1 is derived from [70] for pre-ranking systems. It aggregates user and movie features from ratings, applies an FFNN model and a "LIKE" filter to select trending movies within a given movie genre, and cross-joins them with users for scoring via a widely used industrial pre-ranking model, two-tower model [97]. Q2 and Q3 are adopted from real-world production pipelines described in the multi-stage recommendation technical blog [98] for building intelligent recommender systems. Q2 applies a movie trending FFNN and a user-interest FFNN to prefilter user-movie pairs, then joins them with movie relevance tag table where an AutoEncoder [99] is used to transform the high-dimensional sparse movie-tag relevance vector into a low-dimensional dense representation. The output is passed to a DLRM [88] to estimate a recommendation score. Q3 uses two FFNN-based filters to identify movies that users may be interested in. For each user-movie pair, a vector similarity search is performed using cosine similarity on the movie’s dense representation, which is derived from the movie’s tag relevance matrix via an AutoEncoder model. This step enhances the recommendation results by retrieving similar movies.

2) *Retailing-Complex Queries*: To evaluate performance on complex, modern model architectures not currently covered by existing benchmarks, we designed three realistic inference queries for trip classification (Q1), credit card fraud detection (Q2), and product recommendation (Q3) using the TPCx-AI dataset [39] with scale-factors 1 and 10. While maintaining the original schema relations, we replaced the simpler default models with production-grade deep learning architectures, including FFNN, XGBoost, and Two-Tower models. Q1 joins the *Order* and *Store* tables, applies a UDF model and filters to remove special or non-popular store orders and Sunday purchases, and sends the results to an FFNN model for trip classification. Q2 aggregates the *Order* table to generate per-customer features, which is joined with the *Transaction* table and the *Customer* table. The resulting features are sent to two fraud transaction prediction models: an *XGBoost model* and a FFNN model. The query returns transactions identified as fraud by *both* models. Q3 first filters and aggregates the *product_rating*. The output is joined with the *product* table to embed text attributes as product vectors. Similar processing is applied to the *customer* table to obtain the customer features. The two are cross-joined and sent to a two-tower model that ranks *product-customer* pairs.

3) *Retailing-Simplified Queries*.: We found baselines such as MADLib, SystemDS, and PostgresML do not natively support first two benchmarks involving complex AI/ML functions and multiple AI/ML filters. For fair comparisons, we evaluated three simplified queries directly adopted from the official TPCx-AI benchmark with scale-factors 1 and 10 for product rating, trip classification, and fraud detection. Q1 applies an SVD model [89] to factorize the *product_rating* matrix for product recommendation. Q2 uses a 50-tree XGBoost model to classify trips based on aggregated features joined from the *store* and *order* tables. Q3 applies logistic regression for fraud detection by joining the features from *financial_account* and *financial_transaction* tables.

4) *Other Analytics Queries*.: We also evaluated three analytics workloads on real-world datasets - fraud detection on the Credit Card dataset [40] (Q1), hotel ranking prediction on Expedia dataset [41] (Q2), and codeshare classification on Flights dataset [41] (Q3), which are widely used for evaluating in-DB ML systems [27], [51]. At initial scale (Scale=1), the ML inputs have around 289,000 rows and 29 features for Q1, 79,000 rows and 3,000 features for Q2, and 7,000 rows and 6,000 features for Q3. We further evaluate them at Scale=10, which increases the row count by 10 $\times$. The Credit Card workload performs a single table scan. Expedia performs a three-way join across *listings*, *hotel*, and *search* tables. Flights performs a four-way join across *routes*, *airlines*, *source*, and *destination airports*. All the workloads apply 4-6 predicate filters, then standardizes numerical and one-hot-encodes categorical columns. The resulting features feed into tree-based classifiers - a single decision tree for Expedia and a 100-tree ensemble (max depth 9) for Credit Card and Flights. Additionally, we include the state-of-the-art IMBridge [51], [55], as a baseline, which focuses on UDF-

level optimization for inference queries. It is excluded from the previous benchmark due to its limited support for complex models used in that settings.

5) *A Large-Scale Randomly Generated Inference Query Benchmark*: We designed 20 inference query templates including 10 for the MovieLens dataset, and 10 for TPCx-AI dataset (Scale-1). To eliminate selection bias, we derived these templates from community and industry standards. For MovieLens, we crawled the most highly-voted data science notebooks from Kaggle [100] and abstracted their logic. These templates cover recommendation, rating prediction, collaborative filtering, and rating analysis, using two-tower recommendation models, DLRM, AutoEncoders, SVD, and FFNN. For TPCx-AI, we adopted the queries from its official use cases, including recommendation, trip classification, fraud and spam detection, customer segmentation, and sales prediction, employing decision forests, SVD, FFNN, logistic regression, and K-Means. Additional details are available in the *supplementary document*. Each sampled query varies in model architecture (e.g., different number of layers and neurons for FFNN models), numbers and selectivity of filter predicates. Following [81], we divide the query templates into two groups: six randomly chosen from both workloads form the out-of-distribution set used solely for evaluation, while the remaining 14 generate both training and evaluation queries under different random seeds. We generated 10,000 queries for training the query embedding and cost estimation models, and an additional 1,000 in-distribution and 1,000 out-of-distribution queries to evaluate our optimizer’s generalization capability.

D. Overall Performance Comparison

TABLE I: End-to-End Latency Comparison of Complex Queries

		O1	O2	O3	O4	Latency
Recommendation Queries	Q1	CACTUSDB	✓		✓	0.15 sec
		EvaDB	✓			42.80 sec
		Apache PySpark w/ UDF	✓			60.28 sec
		DL-Centric			✓	48.88 sec
	Q2	CACTUSDB	✓	✓	✓	3.35 sec
		EvaDB	✓			Failed (OOM)
		Apache PySpark w/ UDF	✓			Failed (OOM)
		DL-Centric			✓	142.96 sec
	Q3	CACTUSDB	✓		✓	6.62 sec
		EvaDB	✓			Failed (OOM)
		Apache PySpark w/ UDF	✓			Failed (OOM)
		DL-Centric			✓	207.12 sec
Retailing-Complex Queries	Q1	CACTUSDB	✓	✓	✓	72.07 sec
		EvaDB	✓			123.24 sec
		Apache PySpark w/ UDF	✓			307.88 sec
		DL-Centric			✓	163.97 sec
	Q2	CACTUSDB	✓		✓	44.38 sec
		EvaDB	✓			275.76 sec
		Apache PySpark w/ UDF	✓			1320.47 sec
		DL-Centric			✓	304.81 sec
	Q3	CACTUSDB	✓		✓	3.46 sec
		EvaDB	✓			119.62 sec
		Apache PySpark w/ UDF	✓			1525.94 sec
		DL-Centric			✓	403.84 sec

Complex Inference Queries. As shown in Tab. I, CACTUSDB significantly outperformed the other systems across all six complex queries. CACTUSDB achieved 22 – 401 $\times$ speedup on recommendation queries, and 1.7 – 441 $\times$ speedup on retailing queries, in terms of end-to-end latency compared to the fastest baseline.

We further analyzed the optimization cost, ranging from 0.02s to 0.35s. The specific overhead ratios for the six queries are 13%, 3.8%, 3.7%, 0.5%, 0.7%, and 0.8%, respectively. Notably, across the entire workload, the optimization phase accounts for approximately 1% of the total cumulative execution

time, demonstrating that the overhead is negligible. We further conducted a detailed analysis of the optimizer performance in Sec. V-E. Beyond latency, we profiled the peak memory usage to evaluate resource efficiency (Fig. 9). CACTUSDB demonstrated superior memory management, peaking at only 14% usage even for heavy workloads. In contrast, baselines suffered from higher memory usage, which can lead to OOM errors when the model is too large, due to their reliance on UDF execution models that require intermediate data materialization. CACTUSDB avoids these bottlenecks through the synergy of Velox-based vectorized execution optimization (e.g., zero-copy and lazy materialization) and our query optimizer, which actively selects plans that minimize intermediate data volumes early.

(1) Recommendation queries. For Q1, CACTUSDB factorized the two-tower model (R4-1, R1-4) and pushed down the user tower and movie tower through the cross-join (R1-3), leading to a significant reduction in overall latency. For Q1, EvaDB achieved the second-best performance. However, it needs to transfer the data from the database to the Python runtime for inferences, which accounted for 16% of the end-to-end time. For Q2 and Q3, CACTUSDB transforms the `matMul` in a large AutoEncoder model (that encodes the high-dimensional sparse tag matrix into a low-dimensional dense representation) to relational processing (R3-1). This optimization greatly reduced memory footprint. In contrast, EvaDB and the UDF-based PySpark implementation struggled with handling large matrix computations, resulting in out-of-memory (OOM) errors. DL-Centric implementation suffers from cross-system feature transfer overhead, which accounts for 37% of end-to-end time.

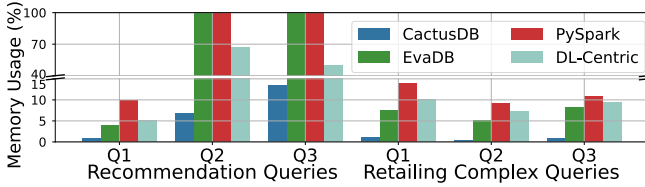


Fig. 9: Memory Usage Comparison for Complex Inference Queries

(2) Retailing-Complex queries. CACTUSDB significantly outperforms all baselines across the three queries, achieving the highest speedup on Q3 (product recommendation). The gain is mainly driven by splitting user-product feature computations (R4-1) and pushdown (R1-3). For trip type classification (Q1) and fraudulent transaction detection (Q2), performance gain stems from factorizing `matMul` (R2-1) and the transformation of XGBoost to `crossJoin` and `aggregate` (R3-2), respectively. Additional improvements come from reordering and pushing down AI/ML filters (R1-1 and R1-2). In contrast, EvaDB and DL-Centric incur high latency from database-Python transfer and costly operators, such as matrix multiplication and XGBoost, while UDF-based PySpark performs worst due to missing UDF optimizations.

Retailing-Simplified Queries. Fig. 10 shows that CACTUSDB consistently achieved the best performance across all three queries at both scale factors, yielding 1.7–20.8× speedups over

the fastest baseline. MADLib outperforms well for Q1 and Q2 at scale factor 1, but it internally relies on Python runtime (e.g., scikit-learn’s SVD and XGBoost) introduces extra overhead. In Q2 with scale factor 10, MADLib introduces additional recursive function calls during XGBoost inference as the data size increases, leading to significant performance degradation. In Q3, its native logistic regression is less efficient than scikit-learn’s highly optimized implementation. For PostgresML, its batch prediction API is significantly faster than its regular prediction API but implemented as an aggregation operator, preventing simultaneous access to non-group-by attributes and causing inefficiency in Q2. It also lacks SVD model support for Q1. SystemDS exhibits slower inference due to less optimized XGBoost and SVD implementations, while DL-Centric suffers from data transfer overhead despite leveraging high-performance ConnectorX library [93].

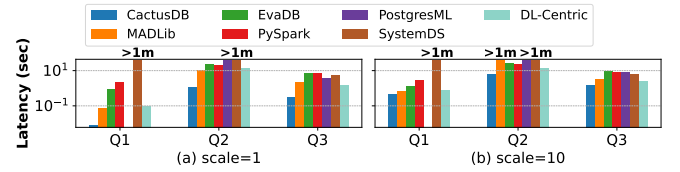


Fig. 10: Latency Comparison for Retailing-Simplified Queries (Log Scale). ">1m" indicates latency greater than 1 minute.

Other Analytics Queries. As shown in Fig. 11, CACTUSDB outperforms the best of other baselines by 2 – 2.5× at scale-1 and 1.2 – 5.7× at scale-10. PostgresML is slightly slower (1.5× and 1.2× slower) than CACTUSDB on the first two queries on scale-10 data, but struggles with the larger Q3 workload that contains many rows, on which it is 66× slower than CACTUSDB. IMBridge benefits from batch processing and context caching but suffers from data transfer overhead between the database query engine and the model inference backend when data size increased. Although Pyspark and SystemDS benefit from their parallel execution and exhibit more stable runtimes, CACTUSDB outperformed PySpark by 1.6 – 9.2× and SystemDS by 1.6 – 9.5× on scale-10 data. These systems all outperformed EvaDB and MADLib on those three analytics workloads.

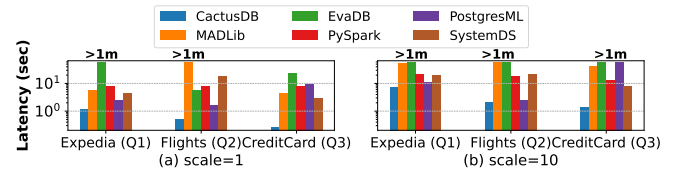


Fig. 11: Latency Comparison for Three Analytics Queries (Log Scale). ">1m" indicates latency greater than 1 minute.

Ablation Study. To understand the benefits of integrating all co-optimization rules, we compare the performance of applying all optimization techniques versus only one category on the recommendation and retailing-complex queries. As shown in Tab. II, integrating all co-optimization rules yields substantially higher speedups than using only one category. For example, Recommendation-Q1 achieves 676.1× speedup over the default plan by pushing down filters and factorizing model computation

to avoid redundant inference.

TABLE II: Ablation Study

Workloads	Queries	Un-optimized	O1	O2	O3	O4	Combined
Recommendation	Q1	1.0X	45.1X	NA	NA	10.9X	676.1X
	Q2	1.0X	3.8X	2.9X	1.8X	2.0X	18.5X
	Q3	1.0X	3.5X	NA	2.1X	1.5X	12.4X
Retailing	Q1	1.0X	1.3X	1.5X	NA	1.2X	1.9X
	Q2	1.0X	1.7X	2.0X	1.5X	1.6X	4.3X
	Q3	1.0X	2.9X	15.8X	NA	2.6X	28.3X

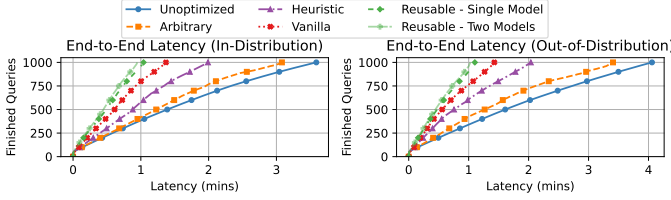


Fig. 12: Query Optimizer Comparison on End-to-End Latency

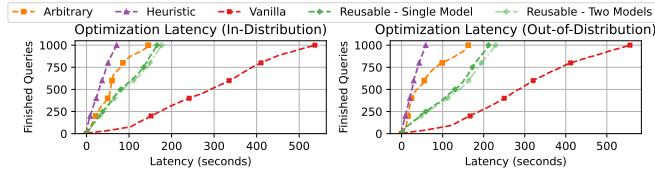


Fig. 13: Query Optimizer Comparison on Optimization Latency

E. Evaluation of Reusable MCTS Optimizer

In this section, we compare our reusable MCTS inference query optimizer with alternative optimization strategies in terms of end-to-end latency and optimization overhead on 1,000 in-distribution (ID) queries and 1,000 out-of-distribution (OOD) queries. Results are reported as elapsed latency versus the number of finished queries in Fig. 12 and Fig. 13. The results revealed that the reusable MCTS effectively balances optimization overheads and execution latency by reusing MCTS search trees across queries.

For ID queries, the reusable MCTS achieves an 89% state collision rate (i.e., the ratio of queries matching existing MCTS states) greatly reducing optimization time compared to vanilla MCTS. While, vanilla MCTS produces high-quality query plans, it incurs substantial optimization overhead due to the need of rebuilding a search tree from scratch for each query, becoming a bottleneck as shown in Fig. 13. The heuristic optimizer performs better than the arbitrary baseline but often produces suboptimal plans due to its fixed rule order.

Our reusable MCTS also generalizes well to OOD queries. Initially, OOD queries require building new MCTS trees due to the absence of reusable states, resulting in higher optimization latency, but optimization efficiency improves as more reusable states are found. On average, the state collision rate reaches 72%, indicating strong generalization. As shown in Fig. 12 and Fig. 13, our two-model strategy introduces minor optimization overhead —6% for ID and 7% for OOD queries. However, the improved query plan latency estimation leads to better optimization results, achieving 10% and 7% improvements in end-to-end execution latency, respectively.

Storage Overhead Analysis The Model2Vec and Query2Vec models occupy 8MB and 23MB, respectively. We incrementally

maintain MCTS search trees, where each node stores a 393-dimensional query plan embedding, along with the visit count, cumulative reward, and the applied optimization rule, requiring about 1.6KB per node. Nearest neighbor retrieval during MCTS search uses a Faiss flat index with cosine similarity, adding no extra storage overhead. Thanks to MCTS tree reusability, the overall storage cost remains low. For instance, optimizing 1,000 queries requires less than 2MB in total.

Further discussion: GPU Support While this work focuses on CPU, our reusable MCTS optimizer is aware of the equivalence between CPU and GPU implementation of the same ML function and leverages R4-2 to invoke LibTorch CUDA library [101] in our GPU environment (see the beginning of Sec. V). As shown in Tab. III, GPU offloading achieved by our optimizer yields an additional 17%–55% performance improvement over the optimized query plan and accelerates model inference by 1.3 $\times$ on average. However, fully optimizing GPU acceleration raises challenges in pipelining, load balancing, and CPU–GPU coordination. These aspects are beyond the scope of this paper and are left for future work.

TABLE III: Recommendation Query Performance with GPU Enabled

	EvaDB	Apache PySpark w/UDF	DL-Centric	CactusDB	
				w/o GPU Extension	w/ GPU Extension
Q1	40.40 sec	48.56 sec	47.2 sec	0.14 sec	0.09 sec
Q2	Failed (OOM)	Failed (OOM)	119.54 sec	2.71 sec	2.14 sec
Q3	Failed (OOM)	Failed (OOM)	172.39 sec	18.67 sec	15.93 sec

VI. CONCLUSION

In this paper, we present CACTUSDB, a system that enables efficient execution and automatic optimization of SQL-ML queries. CACTUSDB is the first database system that supports applying diverse co-optimization techniques to different sub-computations of one inference query by introducing a flexible and expressive three-level IR. To efficiently explore the enlarged search space resulting from the integration of co-optimization techniques, we propose a novel, reusable MCTS-based optimizer co-designed with our three-level IR. Unlike existing MCTS-based query optimizers, ours represents query plans as embedding vectors (states) and co-optimization techniques as configurable actions, enabling MCTS state sharing across queries. In addition, the action configuration process provides opportunities to use heuristics to match a rule, as chosen by MCTS, to a sub-computation, further accelerating the search.

CACTUSDB achieved 1.7 $\times$ to 441 $\times$ speedup over the fastest baseline systems across 11 inference workloads. Evaluation on 2,000 inference queries further demonstrates that our reusable MCTS optimizer achieved better effectiveness-efficiency trade-offs than alternative optimizers.

Limitations and Future Works First, CACTUSDB currently lacks a user-friendly front-end for converting existing Python-based data science pipelines into logical query plans. As a result, integrating legacy workflows requires manual effort and familiarity with the system internals. Second, similar to Raven [27], the CACTUSDB query optimizer relies on a predefined set of machine learning functions and rewrite rules to support various co-optimization techniques. Introducing new co-optimization strategies that involve novel AI/ML functions requires additional engineering effort to implement

the corresponding functions and explicitly inform the optimizer of the associated rewrite actions. Third, while this work demonstrates that many popular models and widely used AI/ML components, such as fully connected neural networks, decision forest models, and two-tower recommendation models, can be transformed into analyzable expressions (i.e., white boxes) for automatic co-optimization with relational processing, the current implementation represents more complex models, such as large language models (LLMs), as black boxes. Consequently, CACTUSDB can only apply co-optimization techniques from the O1 category to these models.

Future work includes developing user-friendly toolchains to facilitate the import of legacy code, extending the query optimizer with new ML functions and rewrite rules, and representing LLMs as white-box models to enable richer and more effective co-optimization techniques.

VII. AI-GENERATED CONTENT ACKNOWLEDGEMENT

No generative AI tools were used to generate the content in this submission, including text, figures, tables, and code.

REFERENCES

- [1] C. Ruan, A. Stewart, H. Li, R. Ye, D. Vengerov, and H. Wang, "Dynamic embedding-based retrieval for personalized item recommendations at instacart," in *Companion Proceedings of the ACM Web Conference 2023*, 2023, pp. 983–987.
- [2] L. Patel, S. Jha, C. Guestrin, and M. Zaharia, "Lotus: Enabling semantic queries with llms over tables of unstructured and structured data," *arXiv preprint arXiv:2407.11418*, 2024.
- [3] C. Liu, M. Russo, M. Cafarella, L. Cao, P. B. Chen, Z. Chen, M. Franklin, T. Kraska, S. Madden, and G. Vitisano, "A declarative system for optimizing ai workloads," *arXiv preprint arXiv:2405.14696*, 2024.
- [4] N. Katkov, "Operationalizing fraud prevention on ibm z16," 2022. [Online]. Available: <https://www.ibm.com/downloads/cas/DOXY3Q94>
- [5] G. Li, J. Sun, L. Xu, S. Li, J. Wang, and W. Nie, "Gaussml: An end-to-end in-database machine learning system," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 5198–5210.
- [6] G. T. Kakkar, J. Cao, P. Chunduri, Z. Xu, S. R. Vyalla, P. Dintyala, A. Prabhakaran, J. Bang, A. Sengupta, K. Ravichandran *et al.*, "Eva: An end-to-end exploratory video analytics system," in *Proceedings of the Seventh Workshop on Data Management for End-to-End Machine Learning*, 2023, pp. 1–5.
- [7] "postgresml," <https://postgresml.org/docs/introduction/getting-started>, 2024.
- [8] A. Eichenberger, Q. Lin, S. Masood, H. Min, A. Sim, J. Wang, Y. Wang, K. Wu, B. Yuan, L. Zhou *et al.*, "Serving deep learning model in relational databases," *arXiv preprint arXiv:2310.04696*, 2023.
- [9] Q. Lin, S. Wu, J. Zhao, J. Dai, M. Shi, G. Chen, and F. Li, "Smartlite: A dbms-based serving system for dnn inference in resource-constrained environments," *Proceedings of the VLDB Endowment*, vol. 17, no. 3, pp. 278–291, 2023.
- [10] H. Shahrokhi, A. Kaboli, M. Ghorbani, and A. Shaikhha, "Pytond: Efficient python data science on the shoulders of databases," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 423–435.
- [11] A. Shaikhha, M. Schleich, and D. Olteanu, "An intermediate representation for hybrid database and machine learning workloads," *Proceedings of the VLDB Endowment*, vol. 14, no. 14 (12), pp. 2831–2834, 2021.
- [12] H. Guan, S. Masood, M. Dwarampudi, V. Gunda, H. Min, L. Yu, S. Nag, and J. Zou, "A comparison of end-to-end decision forest inference pipelines," in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, 2023, pp. 200–215.
- [13] G. J. Annas, "Hippa regulations: a new era of medical-record privacy?" *New England Journal of Medicine*, vol. 348, p. 1486, 2003.
- [14] P. Regulation, "General data protection regulation," *Intouch*, vol. 25, pp. 1–5, 2018.
- [15] "The total cost of ownership (tco) of amazon sagemaker," 2022, accessed: 2024-12-20. [Online]. Available: <https://pages.awscloud.com/NAME-In-GC-400-machine-learning-sagemaker-tco-learn-ty.html>
- [16] D. Crankshaw, "The design and implementation of low-latency prediction serving systems," Ph.D. dissertation, UC Berkeley, 2019.
- [17] J. M. Hellerstein, C. Ré, F. Schoppmann, D. Z. Wang, E. Fratkin, A. Gorajek, K. S. Ng, C. Welton, X. Feng, K. Li *et al.*, "The madlib analytics library," *Proceedings of the VLDB Endowment*, vol. 5, no. 12, 2012.
- [18] J. Yang, X. Yi, D. Zhiyuan Cheng, L. Hong, Y. Li, S. Xiaoming Wang, T. Xu, and E. H. Chi, "Mixed negative sampling for learning two-tower neural networks in recommendations," in *Companion proceedings of the web conference 2020*, 2020, pp. 441–447.
- [19] M. Schleich, D. Olteanu, and R. Ciucanu, "Learning linear regression models over factorized joins," in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 3–18.
- [20] L. Chen, A. Kumar, J. Naughton, and J. M. Patel, "Towards linear algebra over normalized data," *Proc. VLDB Endow.*, vol. 10, no. 11, p. 1214–1225, Aug. 2017. [Online]. Available: <https://doi.org/10.14778/3137628.3137633>
- [21] S. Li, L. Chen, and A. Kumar, "Enabling and optimizing non-linear feature interactions in factorized linear algebra," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 1571–1588.
- [22] M. Schleich, D. Olteanu, M. Abo Khamis, H. Q. Ngo, and X. Nguyen, "A layered aggregate engine for analytics workloads," in *Proceedings of the 2019 International Conference on Management of Data*, ser. SIGMOD '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1642–1659. [Online]. Available: <https://doi.org/10.1145/3299869.3324961>
- [23] B. Yuan, D. Jankov, J. Zou, Y. Tang, D. Bourgeois, and C. Jermaine, "Tensor relational algebra for distributed machine learning system design," *Proc. VLDB Endow.*, vol. 14, no. 8, pp. 1338–1350, 2021. [Online]. Available: <http://www.vldb.org/pvldb/vol14/p1338-yuan.pdf>
- [24] D. Jankov, S. Luo, B. Yuan, Z. Cai, J. Zou, C. Jermaine, and Z. J. Gao, "Declarative recursive computation on an rdbms," *Proceedings of the VLDB Endowment*, vol. 12, no. 7, 2019.
- [25] S. Palkar, J. J. Thomas, A. Shanbhag, D. Narayanan, H. Pirk, M. Schwarzkopf, S. Amarasinghe, M. Zaharia, and S. InfoLab, "Weld: A common runtime for high performance data analytics," in *Conference on Innovative Data Systems Research (CIDR)*, 2017, p. 45.
- [26] S. Palkar, J. Thomas, D. Narayanan, P. Thaker, R. Palamuttam, P. Negi, A. Shanbhag, M. Schwarzkopf, H. Pirk, S. Amarasinghe *et al.*, "Evaluating end-to-end optimization for data analytics applications in weld," *Proceedings of the VLDB Endowment*, vol. 11, no. 9, pp. 1002–1015, 2018.
- [27] K. Park, K. Saur, D. Banda, R. Sen, M. Interlandi, and K. Karanasos, "End-to-end optimization of machine learning prediction queries," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 587–601.
- [28] M. Schleich, A. Shaikhha, and D. Suci, "Optimizing tensor programs on flexible storage," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–27, 2023.
- [29] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen *et al.*, "Mllib: Machine learning in apache spark," *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1235–1241, 2016.
- [30] F. Del Buono, M. Paganelli, P. Sottovia, M. Interlandi, and F. Guerra, "Transforming ml predictive pipelines into sql with masq," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2696–2700.
- [31] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of monte carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in games*, vol. 4, no. 1, pp. 1–43, 2012.
- [32] X. Zhou, G. Li, C. Chai, and J. Feng, "A learned query rewrite system using monte carlo tree search," *Proceedings of the VLDB Endowment*, vol. 15, no. 1, pp. 46–58, 2021.
- [33] X. Yu, C. Chai, G. Li, and J. Liu, "Cost-based or learning-based? a hybrid query optimizer for query plan selection," *Proceedings of the VLDB Endowment*, vol. 15, no. 13, pp. 3924–3936, 2022.

- [34] Q. Bai, S. Alsudais, C. Li, and S. Zhao, "Maliva: Using machine learning to rewrite visualization queries under time constraints." in *EDBT*, 2023, pp. 157–170.
- [35] S. Sikdar and C. Jermaine, "Monsoon: Multi-step optimization and execution of queries with partially obscured predicates," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 225–240.
- [36] P. Pedreira, O. Erling, M. Basmanova, K. Wilfong, L. Sakka, K. Pai, W. He, and B. Chattopadhyay, "Velox: meta's unified execution engine," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3372–3384, 2022.
- [37] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *Acm transactions on interactive intelligent systems (tiis)*, vol. 5, no. 4, pp. 1–19, 2015.
- [38] M. Sarwat, R. Moraffah, M. F. Mokbel, and J. L. Avery, "Database system support for personalized recommendation applications," in *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. IEEE, 2017, pp. 1320–1331.
- [39] C. Brücke, P. Härtling, R. D. E. Palacios, H. Patel, and T. Rabl, "Tpcx-ai-an industry standard benchmark for artificial intelligence and machine learning systems," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3649–3661, 2023.
- [40] "Kaggle credit card fraud dataset," 2013, accessed: 2025-07-15. [Online]. Available: <https://www.kaggle.com/mlg-ulb/creditcardfraud>
- [41] Project Hamlet, "Project hamlet," <https://adalabucsd.github.io/hamlet.html>, 2021, accessed: 2025-07-15.
- [42] G. T. D. Group, "Evadb optimization," <https://evadb.readthedocs.io/en/stable/source/reference/optimizations.html>, 2023, accessed: 2025-01-03.
- [43] S. Luo, Z. J. Gao, M. Gubanov, L. L. Perez, and C. Jermaine, "Scalable linear algebra on a relational database system," *ACM SIGMOD Record*, vol. 47, no. 1, pp. 24–31, 2018.
- [44] M. Boehm, M. W. Dusenberry, D. Eriksson, A. V. Evfimievski, F. M. Manshadi, N. Pansare, B. Reinwald, F. R. Reiss, P. Sen, A. C. Surve *et al.*, "Systemml: Declarative machine learning on spark," *Proceedings of the VLDB Endowment*, vol. 9, no. 13, pp. 1425–1436, 2016.
- [45] L. Zhou, J. Chen, A. Das, H. Min, L. Yu, M. Zhao, and J. Zou, "Serving deep learning models with deduplication from relational databases," *Proc. VLDB Endow.*, vol. 15, no. 10, pp. 2230–2243, 2022. [Online]. Available: <https://www.vldb.org/pvldb/vol15/p2230-zou.pdf>
- [46] A. Kunft, A. Katsifodimos, S. Schelter, S. Breß, T. Rabl, and V. Markl, "An intermediate representation for optimizing machine learning pipelines," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1553–1567, 2019.
- [47] A. Shaikhha, M. Huot, J. Smith, and D. Olteanu, "Functional collection programming with semi-ring dictionaries," *Proceedings of the ACM on Programming Languages*, vol. 6, no. OOPSLA1, pp. 1–33, 2022.
- [48] A. Vasudevan, A. Anderson, and D. Gregg, "Parallel multi channel convolution using general matrix multiplication," in *2017 IEEE 28th international conference on application-specific systems, architectures and processors (ASAP)*. IEEE, 2017, pp. 19–24.
- [49] "Implementation of convolution as matrix multiplication," 2015, accessed: 2024-12-15. [Online]. Available: <https://cs231n.github.io/convolutional-networks/>
- [50] "Optimizing tensorflow convolution performance on aarch64," 2016, archived at <https://web.archive.org/web/20211127091024/https://www.linaro.org/blog/optimizing-tensorflow-convolution-performance-on-aarch64>.
- [51] C. Zhang, J. Peng, C. Xu, Q. Xu, and C. Yang, "Imbridge: Impedance mismatch mitigation between database engine and prediction query execution," in *Companion of the 2024 International Conference on Management of Data*, 2024, pp. 456–459.
- [52] P. Singh and P. Singh, "Manage data with pyspark," *Machine Learning with PySpark: With Natural Language Processing and Recommender Systems*, pp. 15–37, 2022.
- [53] A. Kumar, J. Naughton, and J. M. Patel, "Learning generalized linear models over normalized data," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 1969–1984. [Online]. Available: <https://doi.org/10.1145/2723372.2723713>
- [54] Z. Huang, R. Sen, J. Liu, and E. Wu, "Joinboost: Grow trees over normalized data using only sql," *Proc. VLDB Endow.*, vol. 16, no. 11, p. 3071–3084, Jul. 2023. [Online]. Available: <https://doi.org/10.14778/3611479.3611509>
- [55] C. Zhang, J. Peng, C. Xu, Q. Xu, and C. Yang, "Mitigating the impedance mismatch between prediction query execution and database engine," *Proc. ACM Manag. Data*, vol. 3, no. 3, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3725326>
- [56] M. Boehm, I. Antonov, S. Baunsgaard, M. Dokter, R. Ginthör, K. Innerebner, F. Klezin, S. Lindstaedt, A. Phani, B. Rath *et al.*, "Systemds: A declarative machine learning system for the end-to-end data science lifecycle," *arXiv preprint arXiv:1909.02976*, 2019.
- [57] T. White, *Hadoop: The Definitive Guide*. O'Reilly Media, 2012.
- [58] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: cluster computing with working sets," in *USENIX HotCloud*, 2010, pp. 1–10.
- [59] M. Paganelli, P. Sottovia, K. Park, M. Interlandi, and F. Guerra, "Pushing ml predictions into dbmss," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 10, pp. 10 295–10 308, 2023.
- [60] M. Jungmair, A. Kohn, and J. Giceva, "Designing an open framework for query optimization and compilation," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2389–2401, 2022.
- [61] M. Jungmair and J. Giceva, "Declarative sub-operators for universal data processing," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3461–3474, 2023.
- [62] C. Latner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: A compiler infrastructure for the end of moore's law," *arXiv preprint arXiv:2002.11054*, 2020.
- [63] H. Pirahesh, J. M. Hellerstein, and W. Hasan, "Extensible/rule based query rewrite optimization in starburst," *ACM Sigmod Record*, vol. 21, no. 2, pp. 39–48, 1992.
- [64] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi *et al.*, "Spark sql: Relational data processing in spark," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 2015, pp. 1383–1394.
- [65] G. Graefe, "The cascades framework for query optimization," *IEEE Data Eng. Bull.*, vol. 18, no. 3, pp. 19–29, 1995.
- [66] Apache Calcite Community, *Apache Calcite: A Dynamic Data Management Framework*, 2024, accessed February 2025. [Online]. Available: <https://calcite.apache.org/>
- [67] The PostgreSQL Global Development Group, "Postgresql," 2025, accessed February 2025. [Online]. Available: <https://www.postgresql.org/>
- [68] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: a learned query optimizer," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019.
- [69] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1275–1288.
- [70] X. Li, B. Chen, H. Guo, J. Li, C. Zhu, X. Long, S. Li, Y. Wang, W. Guo, L. Mao *et al.*, "Inttower: the next generation of two-tower model for pre-ranking system," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 3292–3301.
- [71] H. S. Chang, M. C. Fu, J. Hu, and S. I. Marcus, "An adaptive sampling algorithm for solving markov decision processes," *Operations Research*, vol. 53, no. 1, pp. 126–139, 2005.
- [72] C. A. Gomez-Urbe and N. Hunt, "The netflix recommender system: Algorithms, business value, and innovation," *ACM Transactions on Management Information Systems (TMIS)*, vol. 6, no. 4, pp. 1–19, 2015.
- [73] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [74] A. Jassy, "Aws re:invent 2018 - keynote," <https://www.youtube.com/watch?v=ZOIkOnW640A>, 2018, accessed: 2024-12-12.
- [75] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *arXiv preprint arXiv:1904.03711*, 2019.
- [76] J. Sun and G. Li, "An end-to-end learning-based cost estimator," *arXiv preprint arXiv:1906.02560*, 2019.
- [77] X. Yu, G. Li, C. Chai, and N. Tang, "Reinforcement learning with tree- lstm for join order selection," in *2020 IEEE 36th international conference on data engineering (ICDE)*. IEEE, 2020, pp. 1297–1308.

- [78] R. Marcus and O. Papaemmanouil, "Deep reinforcement learning for join order enumeration," in *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*, 2018, pp. 1–4.
- [79] B. Ding, S. Das, R. Marcus, W. Wu, S. Chaudhuri, and V. R. Narasayya, "Ai meets ai: Leveraging query executions to improve index recommendations," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 1241–1258.
- [80] Y. Zhao, G. Cong, J. Shi, and C. Miao, "Queryformer: A tree transformer model for query plan representation," *Proceedings of the VLDB Endowment*, vol. 15, no. 8, pp. 1658–1670, 2022.
- [81] Y. Zhao, Z. Li, and G. Cong, "A comparative study and component analysis of query plan representation techniques in ml4db studies," *Proceedings of the VLDB Endowment*, vol. 17, no. 4, pp. 823–835, 2023.
- [82] Z. Cai, X. Li, B. Hui, M. Yang, B. Li, B. Li, Z. Cao, W. Li, F. Huang, L. Si *et al.*, "Star: Sql guided pre-training for context-dependent text-to-sql parsing," *arXiv preprint arXiv:2210.11888*, 2022.
- [83] T. Gao, X. Yao, and D. Chen, "Simcse: Simple contrastive learning of sentence embeddings," *arXiv preprint arXiv:2104.08821*, 2021.
- [84] S. Chen, J. Fan, B. Wu, N. Tang, C. Deng, P. Wang, Y. Li, J. Tan, F. Li, J. Zhou *et al.*, "Automatic database configuration debugging using retrieval-augmented language models," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–27, 2025.
- [85] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels," *Journal of Machine Learning Research*, vol. 12, no. 9, 2011.
- [86] J. Hostetler, A. Fern, and T. Dietterich, "State aggregation in monte carlo tree search," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, no. 1, 2014.
- [87] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [88] M. Naumov, D. Mudigere, H.-J. M. Shi, J. Huang, N. Sundaraman, J. Park, X. Wang, U. Gupta, C.-J. Wu, A. G. Azzolini *et al.*, "Deep learning recommendation model for personalization and recommendation systems," *arXiv preprint arXiv:1906.00091*, 2019.
- [89] M. E. Wall, A. Rechtsteiner, and L. M. Rocha, "Singular value decomposition and principal component analysis," in *A practical approach to microarray data analysis*. Springer, 2003, pp. 91–109.
- [90] T. Siddiqui, A. Jindal, S. Qiao, H. Patel, and W. Le, "Cost models for big data query processing: Learning, retrofitting, and our findings," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 99–113.
- [91] A. Spark, "Pyspark," <https://spark.apache.org/docs/latest/api/python/index.html>, 2024, accessed: 2025-01-03.
- [92] Databricks, "Model inference using pytorch," <https://docs.databricks.com/en/machine-learning/model-inference/resnet-model-inference-pytorch.html>, 2023, accessed: 2025-01-03.
- [93] X. Wang, W. Wu, J. Wu, Y. Chen, N. Zrymiak, C. Qu, L. Flokas, G. Chow, J. Wang, T. Wang *et al.*, "Connectorx: accelerating data loading from databases to dataframes," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2994–3003, 2022.
- [94] S. Wooders, X. Mo, A. Narang, K. Lin, I. Stoica, J. M. Hellerstein, N. Crooks, and J. E. Gonzalez, "Ralf: Accuracy-aware scheduling for feature store maintenance," *Proceedings of the VLDB Endowment*, vol. 17, no. 3, pp. 563–576, 2023.
- [95] C. Qin, M. Torres, and F. Rusu, "Scalable asynchronous gradient descent optimization for out-of-core models," *Proceedings of the VLDB Endowment*, vol. 10, no. 10, pp. 986–997, 2017.
- [96] M. Ghorbani and A. Shaikhha, "Demonstration of opendbml, a framework for democratizing in-database machine learning," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3970–3973, 2023.
- [97] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, and L. Heck, "Learning deep structured semantic models for web search using clickthrough data," in *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*, 2013, pp. 2333–2338.
- [98] (2024) Deploying a multi-stage recommender system. NVIDIA Merlin. Accessed: 2026-01-08. [Online]. Available: <https://nvidia-merlin.github.io/Merlin/stable/examples/Building-and-deploying-multi-stage-RecSys/index.html>
- [99] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [100] S. Claudia, "Movielens dataset," <https://www.kaggle.com/datasets/sherinclaudia/movielens>, 2019, accessed: 2025-05-17.
- [101] T. P. Foundation, *LibTorch: PyTorch C++ API*, 2018. [Online]. Available: <https://pytorch.org/cppdocs/>
- [102] "Tpcx-ai express benchmark," 2021. [Online]. Available: <https://www.tpc.org/tpcx-ai/default5.asp>

A. Weisfeiler-Lehman Encoding Process

In this section, we describe how the Weisfeiler-Lehman (WL) kernel is applied to construct positive and negative training samples. The WL kernel encoding procedure is outlined in Algorithm 6. It iteratively generates feature vectors by aggregating and updating node labels. While the encoding processes of Model2Vec and Query2Vec are largely similar, they differ in how the initial node labels are assigned. In Model2Vec, the initial labels are determined by the kernel’s FLOPs values. In contrast, Query2Vec assigns initial labels based on different strategies tailored to each node type. These initialization procedures are detailed in Algorithm 7 and Algorithm 9, respectively.

Algorithm 6: Weisfeiler-Lehman Kernel Feature Extraction

```

Input :  $root$ : root node of the model tree
          $T$ : number of WL iterations
Output :  $\mathcal{F}$ : WL subtree feature counts
1  $\mathcal{L}, \mathcal{H}, \mathcal{N} \leftarrow \emptyset$ ; // Label map, history, node list
2  $\text{DFS}(root, \mathcal{L}, \mathcal{H}, \mathcal{N})$ ; // Initialize the labels
   through DFS
3 for  $i \leftarrow 1$  to  $T$  do
4    $\mathcal{L}_{new} \leftarrow \emptyset$ ;
5   foreach  $node \in \mathcal{N}$  do
6      $C \leftarrow \text{sorted}([\mathcal{L}[c] \mid c \in node.children]);$ 
7      $label_{new} \leftarrow \mathcal{L}[node] + C$ ;
8      $\mathcal{L}_{new}[node] \leftarrow label_{new}$ ;
9      $\mathcal{H}[node].append(label_{new})$ ;
10   $\mathcal{L} \leftarrow \mathcal{L}_{new}$ ;
11  $\mathcal{F} \leftarrow \emptyset$ ; // Feature vector
12 foreach  $node \in \mathcal{N}$  do
13   foreach  $label \in \mathcal{H}[node]$  do
14      $\mathcal{F}[label] += 1$ ;
15 return  $\mathcal{F}$ 

```

Algorithm 7: Initial Node Label Assignment for Model2Vec

```

Input :  $node$ : a tree node with operator type and FLOPs
          $\mathcal{G}$ : label groups by kernel type
          $\rho$ : FLOPs range threshold
Output : label: unique label assigned to the node
1  $type \leftarrow node.ml\_op\_type$ ;
2  $flops \leftarrow node.ml\_op\_flops$ ;
3  $group \leftarrow \mathcal{G}[type]$ ;
4  $idx \leftarrow \text{BinarySearchWithRange}(group, flops, \rho)$ ;
5 if  $idx = -1$  then
6    $label \leftarrow t\_f$ ;
7    $group.append(flops)$ ;
8    $group.sort()$ ;
9    $\mathcal{G}[type] \leftarrow group$ ;
10 else
11    $label \leftarrow type + group[idx]$ ;
12 return  $label$ 

```

B. Error Bound Analysis via (p, q) -Consistency

This section provides an error-bound analysis for CAC-TUSDB’s reusable MCTS optimizer using the (p, q) -consistency abstraction framework (as in state-abstraction

Algorithm 8: DFS Traversal for Label Initialization

```

Input :  $root$ : root node of the model tree
          $\mathcal{L}$ : node label map
          $\mathcal{H}$ : node label history
          $\mathcal{N}$ : list of all nodes
Output : Updated  $\mathcal{L}$ ,  $\mathcal{H}$ , and  $\mathcal{N}$ 
1 Function  $\text{DFS}(node, \mathcal{L}, \mathcal{H}, \mathcal{N})$ :
2    $label \leftarrow \text{InitialNodeLabel}(node)$ ;
3    $\mathcal{L}[node] \leftarrow label$ ;
4    $\mathcal{H}[node] \leftarrow [label]$ ;
5    $\mathcal{N}.append(node)$ ;
6   foreach  $child \in node.children$  do
7      $\text{DFS}(child, \mathcal{L}, \mathcal{H}, \mathcal{N})$ ;

```

Algorithm 9: Initial Node Label Assignment for Query2Vec

```

Input :  $node$ : a tree node with type and model embedding
          $\mathcal{G}$ : label groups by node type
          $\mathcal{I}$ : FAISS model embedding index
          $\tau$ : similarity threshold
Output : label: label assigned to the node
1  $type \leftarrow node.nodeType$ ;
2  $group \leftarrow \mathcal{G}[t]$ ;
3 if  $node.hashML$  then
4    $q \leftarrow \text{normalize}(e)$ ;
5    $idx, sim \leftarrow \mathcal{I}.search(node.mlEmbed)$ ;
   // Assign label based on embedding
   similarity
6   if  $sim > \tau$  then
7      $label \leftarrow type + idx$ 
8   else
9      $label \leftarrow type + next\_idx$ ;
10     $\mathcal{I}.add(node.mlEmbed)$ ;
11     $next\_idx \leftarrow next\_idx + 1$ ;
   // Assign label based on relation data
12 else if  $type = \text{TableScanNode}$  then
13    $label \leftarrow t + node.table\_id$ ;
   // Assign label based on filter predicate
14 else if  $type = \text{FilterNode}$  then
15    $c \leftarrow node.filter["colId"]$ ;
16    $o \leftarrow node.filter["opId"]$ ;
17    $v \leftarrow node.filter["val"]$ ;
18    $label \leftarrow t + c + o + v$ ;
   // Assign label based on project expression
19 else if  $type = \text{ProjectNode}$  then
20    $label \leftarrow t + node.projectExpr$ ;
   // Assign label based on join predicate
21 else if  $type \in \{\text{HashJoinNode}, \text{NestedLoopJoinNode}\}$  then
22    $label \leftarrow t + node.join$ ;
   // Assign label based on aggregation keys
23 else if  $type = \text{AggregationNode}$  then
24    $label \leftarrow type + node.aggKeys$ ;
25 else
26    $label \leftarrow type$ ;
27  $group.add(label)$ ;
28 return  $label$ 

```

analyses of MCTS). Unlike the metric/Lipschitz analysis, the (p, q) framework reasons about the quality of an *implicit partition* of states induced by subtree reuse.

1) *Planning Model and Simple Regret*: We model query optimization as a discounted MDP $M = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where states are query plans and actions are configurable rewrite-rule instantiations. Let $V^*(s)$ and $Q^*(s, a)$ be optimal values, and

define the (root) simple regret for the input state s_0 :

$$\mathcal{R}(s_0) = V^*(s_0) - Q^*(s_0, \hat{a}) = \max_{a \in \mathcal{A}(s_0)} Q^*(s_0, a) - Q^*(s_0, \hat{a}), \quad (7)$$

where $\hat{a}$ is the action returned by CACTUSDB after B_{iter} simulations.

C. Proof of Theorem ??

1) Expectimax-tree abstraction and (p, q) -consistency:

We briefly restate the abstract expectimax-tree construction (specialized to our notation). Let $\mathcal{H}$ be the history partition induced by f , and let $\mu = \{\mu_h\}$ be a family of distributions over ground trajectories $t \in h$. The induced abstract-tree transition is

$$\mathcal{P}_\mu(h, a, h') = \sum_{t \in h} \mu_h(t) \sum_{t' \in h'} \mathcal{P}(t, a, t'),$$

and the abstract value functions satisfy the standard Bellman optimality equations on the history tree:

$$Q_\mu^*(h, a) = \mathcal{R}_\mu(h) + \sum_{h' \in \mathcal{H}} \mathcal{P}_\mu(h, a, h') V_\mu^*(h'), \quad (8)$$

$$V_\mu^*(h) = \max_{a \in \mathcal{A}} Q_\mu^*(h, a), \quad (9)$$

where $\mathcal{R}_\mu(h) = \sum_{t \in h} \mu_h(t) \mathcal{R}(t)$ (see [86]).

We assume (p, q) -consistency, i.e., for each $h \in \mathcal{H}$ there exists a_h^* such that (??)–(??) hold. This matches the definition in [86].

2) *Optimal weighting μ^* and one-step divergence:* In expectimax-tree abstractions, correctness depends not only on f but also on how probability mass is assigned to the ground trajectories inside each history. Hostetler et al. define an *optimal* weighting μ^* and measure deviation from it via a one-step divergence quantity; we reuse that notion here. Let $m \triangleq \max_{h \in \mathcal{H}} \delta(\mu, h) \leq 1$ be the maximum single-step divergence (notation and formal definition as in [86]). When reusable MCTS/UCT aggregates statistics by sampling ground transitions and mapping them to histories, it behaves as if it were using μ^* in the abstract tree (under the conditions discussed in [86]); in the most conservative setting we only assume $m \leq 1$.

3) *Key error decomposition and induction:* Fix any history node h and action a . Define the action-value estimation error

$$E(h, a) \triangleq \left| Q_\mu^*(h, a) - \sum_{t \in h} \mu_h(t) Q^*(t, a) \right|.$$

Following the proof strategy in [86], decompose the error into (i) error propagated from deeper abstract values and (ii) error introduced by aggregating/weighting at the current step:

$$E(h, a) \leq E_Q(h, a) + E_\delta(h, a),$$

where E_Q captures the mismatch between $V_\mu^*(\cdot)$ and the weighted ground optimal values below children, and E_δ captures the single-step weighting/aggregation deviation at the current expansion.

The proof proceeds by induction on the remaining depth k to leaves (base case holds at terminal/absorbing nodes). Assume the inductive hypothesis

$$E(h', a) \leq k(p + mq) \quad \text{for all children } h' \text{ of } h \text{ and all } a \in \mathcal{A}.$$

Using $(p, \cdot)$ -consistency (which allows exchanging max and $\sum$ with additive penalty p) plus the inductive hypothesis yields

$$E_Q(h, a) \leq k(p + mq) + p.$$

This corresponds to the step “exchange max and sum at cost $\leq p$ ” in [86].

For the single-step abstraction error, $(\cdot, q)$ -consistency implies that within any child history the optimal values vary by at most q , and the deviation of μ from μ^* scales this by at most the one-step divergence. This yields

$$E_\delta(h, a) \leq mq,$$

as in the convex-combination argument in [86].

Combining gives the inductive step:

$$E(h, a) \leq (k + 1)(p + mq).$$

Thus at the root history h_0 (depth d), for any action a we have $E(h_0, a) \leq d(p + mq)$.

4) *From value-estimation error to root action suboptimality:* Let $a^\# \in \arg \max_a Q_\mu^*(h_0, a)$ be the abstract-tree optimal action, and let $a^* \in \arg \max_a Q^*(s_0, a)$ be the ground-tree optimal root action. In the worst case, the abstract estimate overvalues $a^\#$ by at most $d(p + mq)$ and undervalues a^* by at most $d(p + mq)$, yielding

$$Q^*(s_0, a^*) - Q^*(s_0, a^\#) \leq 2d(p + mq).$$

This is exactly the final root-step argument in [86]. Since $m \leq 1$, we also obtain the simplified bound $2d(p + q)$. $\square$

D. IR Manipulation APIs.

In this section, we present a set of APIs abstracted for manipulating the three-level IR. First, each level of the IR graphs can be manipulated using simple APIs for the addition/removal of a node and the addition/removal of an IR mapping between a top-level IR node and a middle-level IR graph, or between a middle-level IR node and a bottom-level IR graph.

```
//Add and Remove a node node from a certain level of IR graph specified as G.
addNode(node: node to be added, parents: a set of existing nodes, from which new edges to node will be added; children: a set of existing nodes, to which new edges from node will be added, G: the IR graph);
removeNode(node: node to be removed, G: the IR graph);

//Add and Remove an IR mapping between a node and an IR graph.
addIRMapping(node: a node to be mapped to a lower-level IR, G: a lower-level IR that describes the node);
removeIRMapping(node, G);
```

Second, building on the above basic APIs, two cross-level IR manipulation APIs are supported as follows:

Algorithm 10: FactorizeIR: Factorize a bottom/middle-level IR graph and synchronize the changes to its upper-level IR graph.

Input: $G = (V, E)$, an IR graph with nodes V and edges E ;
 $P = \{G^i = (V^i, E^i)\} (i = 1, \dots, k)$: a factorization plan for splitting G into k (factorized) subgraphs, with each containing a set of nodes $V^i \subset V$ and a set of edges $E^i \subset E$; G_U ; v_U^G

```

1 for  $i \in \{1, \dots, k\}$  do
2    $v_U^{G^i} \leftarrow \text{new node}()$ ;  $\text{parents} \leftarrow \Phi$ ;  $\text{children} \leftarrow \Phi$ ;
3   for each source node  $v$  of  $G$  with  $v \in G^i$  do
4      $v_U \leftarrow \text{parent node of } v_U^G \text{ in } G_U \text{ corresponding to } v$ ;
5      $\text{parents} \leftarrow \text{parents} \cup \{v_U\}$ ;
6   for each sink node  $v$  of  $G$  with  $v \in G^i$  do
7      $v_U \leftarrow \text{child node of } v_U^G \text{ in } G_U \text{ corresponding to } v$ ;
8      $\text{children} \leftarrow \text{children} \cup \{v_U\}$ ;
9   for  $j \in \{1, \dots, i-1\}$  do
10    if  $G^j$  has edge pointing to  $G^i$  in  $G$  then
11       $\text{parents} \leftarrow \text{parents} \cup \{v_U^{G^j}\}$ ;
12    if  $G^i$  has edge pointing to  $G^j$  in  $G$  then
13       $\text{children} \leftarrow \text{children} \cup \{v_U^{G^j}\}$ ;
14     $\text{addNode}(v_U^{G^i}, \text{parents}, \text{children}, G_U)$ ;
15     $\text{addIRMapping}(v_U^{G^i}, G^i)$ ;
16 removeNode( $v_U^G, G_U$ ); removeIRMapping( $v_U^G, G$ );
17 return;
```

//factorize an IR graph G into k separate IR graphs $G_1, \dots, G_k$, and factorize the node v_U^G at the upper-level IR graph (G_U) that maps to G into k separate nodes.

factorizeIR(G : the IR graph to be factorized, which should be either a middle-level IR or a bottom-level IR; P : a factorization plan that consists of k factorized graphs, G_U : the upper-level IR of G , v_U^G : the node corresponding to G in G_U);

//reverse process of factorizeIR.

mergeIR($\{G_1, \dots, G_k\}$: graphs to be merged; G_U : upper-level IR graph (middle or top-level IR); $\{v_U^{G_1}, \dots, v_U^{G_k}\}$: the corresponding nodes representing $G_1, \dots, G_k$ in G_U , v_U^{new} : the merged node at G_U);

The above APIs are used to implement techniques R1-2, R1-4, R2-1 to R2-3, and R4-1. For example, the twoTowerModel's bottom-level IR in Fig. ?? (denoted as G) is factorized into three bottom-level IRs (denoted as $P = \{G^{cosSim}, G^{movieTower}, G^{userTower}\}$) by applying R4-1 as shown in Fig. ??a. As part of the factorization, the twoTowerModel node (denoted as v) in the middle-level IR corresponding to Project3 in Fig. ?? (denoted as G_U), is also factorized into three nodes in Fig. ??a: cosSim, movieTower, and userTower. The optimizer implements this process by simply invoking *factorizeIR*(G, P, G_U, v).

factorizeIR() and *mergeIR*() are implemented as two-level graph splitting/merging processes, each of which having a complexity of $O(k^2)$, where k is the number of factorized graphs. The algorithms are engineering-centric and are formalized in Alg. 10 and Alg. 11.

E. Vanilla MCTS Algorithm

The Alg. 12 describes how the vanilla MCTS algorithm is used in query optimization.

Algorithm 11: MergeIR: Merge k (bottom- or middle-level) IR graphs and synchronize changes to the upper-level IR.

Input: $\{G^1, \dots, G^k\}$: k graphs to be merged; G_U ;
 $\{v_U^{G^1}, \dots, v_U^{G^k}\}$: the corresponding nodes representing $G^1, \dots, G^k$ in G_U ; v_U^{new}

```

/* Merge  $G^i$  into  $G^j$  if  $v_U^{G^i}$  and  $v_U^{G^j}$  are connected. */
1 for  $i, j = 1, \dots, k \wedge i \neq j$  do
2   if  $v_U^{G^i}$  has edge  $e$  pointing to  $v_U^{G^j}$  then
3      $v_i \leftarrow e$ 's destination node in  $G^i$ ;
4      $v_j \leftarrow e$ 's destination node in  $G^j$ ;
5      $\text{addEdge}(v_i, v_j)$ ;
/* merge nodes in upper-level IR */
6  $\text{parents} \leftarrow \Phi$ ;  $\text{children} \leftarrow \Phi$ ;
7 for  $i = 1, \dots, k$  do
8    $\text{parents} \leftarrow \text{parents} \cup \text{the set of parent nodes of } v_U^{G^i}$ ;
9    $\text{children} \leftarrow \text{children} \cup \text{the set of children nodes of } v_U^{G^i}$ ;
10   $\text{removeNode}(v_U^{G^i}, G_U)$ ;
11  $\text{addNode}(v_U^{new}, \text{parents}, \text{children}, G_U)$ ;
12  $\text{removeNode}(v_U^G, G_U)$ ;
13  $\text{addIRMapping}(v_U^{new} \rightarrow G_{new} = \cup_{i=1}^k G^i)$ ;
14 return;
```

Algorithm 12: General MCTS Algorithm

Input: *root*, the state of the original query plan; $B_{iteration}$, the budget of MCTS training iterations

```

1 for  $i$  in range( $B_{iteration}$ ) do
2    $\text{node} \leftarrow \text{root}$ ;
3   /* If the current node is fully expanded, perform node selection. Otherwise, perform node expansion. */
4   while  $\text{node.isTerminalState}() == \text{False}$  do
5     if  $\text{node.expanded}$  then
6        $\text{node} \leftarrow \text{select}(\text{node})$ ;
7     else
8        $\text{node} \leftarrow \text{expand}(\text{node})$ ;
9        $\text{node}' \leftarrow \text{rollout}(\text{node})$ ;
10      break;
11    $\text{reward} \leftarrow \text{computeReward}(\text{node}')$ ;
12    $\text{backpropagate}(\text{node}, \text{reward})$ ;
13 return;
```

F. Recommendation Queries

The three recommendation queries we used in evaluation are described in List. 1 based on the MovieLens dataset, with the corresponding ER diagram shown in Fig. 14.

In Q1, the trending model has an input size of 3, with hidden layers of 128 and 64 units. ReLU is used as the activation function. The output layer consists of a single neuron with a sigmoid function. The two tower model initialized embedding tables of dimension 32 for each categorical feature. Later, the concatenated user and movie features are then passed to a fully connected neural network with the two hidden layers, each containing 300 neurons. The output layer consists of 128 neurons. The input features dimensions are 129 for each user and 65 for each movie.

In Q2, the trending model is the same as the one used in Q1. The *predict_user_interest_DNN* is a fully connected neural network with an input dimension of 259 and a single hidden layer containing 128 neurons. Its output consists of two

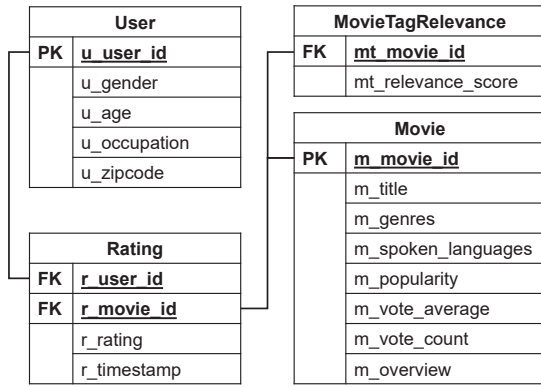


Fig. 14: ER Diagram of Movielens Database

neurons with a softmax activation function. The Autoencoder model takes an input of size 140,979, representing the number of unique movie tags, with a hidden layer of 2,048 neurons. The output is a dense layer with 256 neurons. The DLRM model initializes embedding tables of dimension 128 for each categorical variable. The numerical features are processed by a one-layer MLP, a fully connected neural network with an input size of 256 and output size of 128. The categorical features' embedding vectors are concatenated with the MLP's output and then passed to another one-layer MLP, with an input size of 256, a hidden layer of 128 neurons, and a single-neuron output layer.

In Q3, autoencoder and predict_user_interest_DNN are both fully connected neural networks with the same architecture as described in Q2. predict_rating_DNN is another fully connected neural network, which has an input dimension of 5, two hidden layers with 512 and 1024 neurons, respectively, and an output layer of size 6.

```

1 // Q1
2 SELECT
3   two_tower_model (user_feature, movie_feature)
4 FROM
5   (
6     SELECT
7       concat(embedding (user_id), embedding (
8         gender), embedding (age), embedding (
9         occupation), user_avg_rating) AS
10      user_feature
11 FROM
12   USER u
13   JOIN (
14     SELECT user_id, average (rating) AS
15       user_avg_rating
16     FROM rating GROUP BY user_id
17   ) ON u.user_id=user_id
18 )
19 CROSS JOIN (
20   SELECT
21     concat(embedding (movie_id), embedding (
22       genres), movie_avg_rating) AS
23     movie_feature
24 FROM
25   movie m
26   JOIN (
27     SELECT movie_id, average (rating)
  
```

```

28 FROM rating GROUP BY movie_id
29 ) ON m.movie_id=movie_id
30 WHERE
31   movie.genres LIKE '%Action%' AND
32   trending_movie_DNN (movie_feature)=1
33 )
34 // Q2
35 SELECT
36   DLRM(user_feature,movie_feature,
37     movie_dense_feature)
38 FROM
39   (SELECT
40     feature_extraction1(*) AS user_feature
41 FROM
42   user
43   CROSS JOIN (
44     SELECT
45       feature_extraction2(movie_feature),
46       autoencoder(movie_relevance_tag) AS
47       movie_dense_feature
48 FROM
49   movie
50   JOIN movie_tag_feature ON movie.
51     m_movie_id=movie_tag_feature.
52     mt_movie_id
53 )
54 WHERE
55   predict_trending_movie_DNN (movie_feature
56     )=1
57   AND predict_user_interest_DNN(
58     user_feature, movie_dense_feature)
59     =1)
60 // Q3
61 SELECT
62   cosine_similarity(movie_dense_feature1,
63     movie_dense_feature2) AS relevant_score
64 FROM
65   (
66     SELECT
67       autoencoder(mt_relevance_score) AS
68       movie_dense_feature1
69 FROM
70   (
71     SELECT
72       preprocess1(user.*) AS user_feature,
73       preprocess2(movie.*) AS
74       movie_feature
75 FROM
76   user
77   CROSS JOIN movie
78   JOIN movie_tag_relevance ON mt.
79     movie_id = m_movie_id
80 )
81 WHERE
82   genres LIKE '%Fiction%'
83   AND predict_user_interest_DNN(
84     user_feature, movie_feature) = 1
85   AND predict_rating_DNN(user_feature,
86     movie_feature) > 3
87 )
88 CROSS JOIN(
89   SELECT
90     autoencoder(mt_relevance_score) AS
91     movie_dense_feature2
  
```

```

74 FROM
75     movie_tag_relevance
76 )

```

Listing 1: Query 1

G. Retailing-Complex Queries

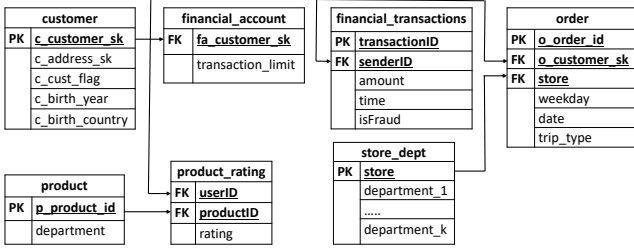


Fig. 15: Simplified ER Diagram of Retailing Dataset from TPCx-AI [102]

List. 2 presents three complex queries based on the TPCx-AI dataset, with the corresponding ER diagram shown in Fig. 15.

In Q1, `is_popular_store` is a UDF that computes a popularity score for each store based on its department availability. `trip_classifier_dnn` is a neural network comprising a customer embedding table and a fully connected neural network. The embedding table is initialized with the dimension of 16, and the resulting customer embedding vector is concatenated with additional features, forming an input of size 63. The input is then passed through a fully connected neural network with two hidden layers of 48 and 24 neurons, respectively, followed by an output layer of size 1000, representing the trip types.

In Q2, `dnn_fraud_predict` is a deep neural network (DNN) model designed for fraud classification, consisting of an input layer of size 32, a single hidden layer with 12 neurons, and an output layer of size 2. `xgboost_fraud_predict` is an XGBoost model with 1,600 trees which take 5 features.

Q3 invokes a two-tower model that initializes multiple embedding tables for categorical variables, each with a dimension of either 16 or 8. Customer features and product features are processed separately through the user tower and product tower. The user tower is a fully connected neural network with an input size of 50, followed by two hidden layers with 128 and 64 neurons and an output layer of 16 neurons. The product tower has a similar structure but with a different input size of 25.

```

1 // Q1
2 SELECT o_order_id, trip_classifier_dnn (concat
3     (order_feature, store_feature))
4 FROM (
5     SELECT o_order_id , get_order_features(
6         o_customer_sk, weekday, date) AS
7         order_feature, store_feature
8     FROM order o

```

```

6 JOIN (
7     SELECT store, concat (*) AS store_feature
8     FROM store_dept_wide s
9 ) ON o. store = s.store
10 WHERE o. weekday != 'Sunday' AND
11     is_popular_store (store_feature) = 1
12 )
13
14 // Q2
15 SELECT transactionID
16 FROM (
17     SELECT c_customer_sk , get_customer_feature
18     (c_address_sk, c_cust_flag, c_birth_year
19     , c_birth_country, transaction_limit) AS
20     customer_feature
21 FROM customer
22 JOIN (
23     SELECT fa_customer_sk, transaction_limit
24     FROM financial_account
25 ) ON c_customer_sk = fa_customer_sk
26 WHERE c_cust_flag = 0
27 JOIN (
28     SELECT transactionID, senderID,
29     get_transaction_feature (*) AS
30     transaction_feature
31 FROM financial_transactions t
32 ) ON c_customer_sk = senderID
33 WHERE age_during_transaction ( c_birth_year
34 , t.time) >= 18 AND is_working_day (t.
35 time) = 1
36 )
37 WHERE xgboost_fraud_predict (concat (
38     customer_feature, transaction_feature)) >=
39     0.5 AND dnn_fraud_predict (concat (
40     customer_feature, transaction_feature)) =
41     1
42
43 // Q3
44 SELECT two_tower_model (customer_feature,
45     product_feature)
46 FROM (
47     SELECT concat (embedding (c_customer_sk),
48     embedding (c_address_sk), c_cust_flag,
49     embedding (get_age (c_birth_year)),
50     embedding (birth_country),
51     customer_avg_rating) AS customer_feature
52 FROM customer c
53 JOIN (
54     SELECT userID, AVG (rating) AS
55     customer_avg_rating
56 FROM product_rating GROUP BY userID
57 ) AS cr ON c. c_customer_sk = cr.userID
58 ) AS customer_outer
59 CROSS JOIN (
60     SELECT concat (embedding (p_product_id),
61     embedding (department),
62     product_avg_rating) AS product_feature
63 FROM product p
64 JOIN (
65     SELECT productID, AVG (rating) AS
66     prod_avg_rating
67 FROM product_rating GROUP BY productID
68 ) AS pr ON p. p_product_id = pr.productID
69 WHERE pr.prod_avg_rating >= 4.0

```

Listing 2: Queries on Retailing Dataset

H. LLM Queries

List. 3 describes the two LLM queries used in our evaluation. We utilize OpenAI’s RESTful API to invoke the gpt-3.5-turbo model to summarize the user/movie information and make the recommendations. The trending_movie_classifier is a fully connected neural network with the same structure as the one described in Sec. F. Q2 employs a RAG-based approach to retrieve movies’ relevant information by performing a nearest-neighbor vector search on pre-built document embedding vectors using Faiss’s cosine similarity index.

```

1 // Q1: Using LLM for Movie Recommendation
2 SELECT LLM('Please give a recommendation score
   and explain why', LLM('Please summarize',
   u.description), LLM('Please summarize', m
   .description))
3 FROM user u CROSS JOIN movie m
4 WHERE m.spoken_language LIKE % English % AND
   trending_movie_classifier(m.popularity, m.
   vote_average, m.vote_num) = True;
5
6 // Q2: Using RAG to retrieve relevant
   information with given movie title and
   using LLM for recommendation
7 SELECT LLM('Please give a recommendation score
   and explain why', LLM('Please summarize',
   u.description), RAG(m.title))
8 FROM user u CROSS JOIN movie m
9 WHERE m.spoken_language LIKE % English % AND
   trending_movie_classifier(m.popularity, m.
   vote_average, m.vote_num) = True;

```

Listing 3: LLM Queries

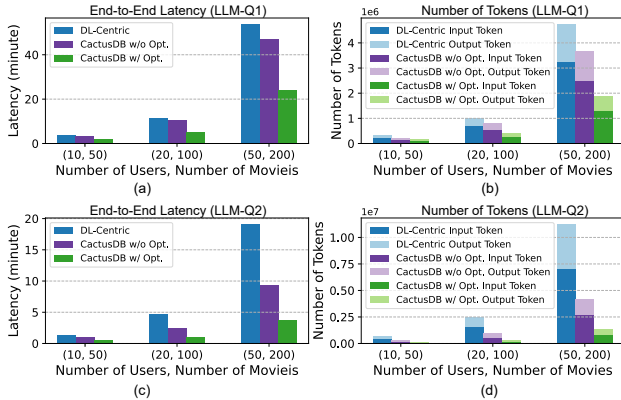


Fig. 16: Evaluating the LLM Queries with R4-1 and R1-3

LLM Inference Queries. In this benchmark, the overheads of LLM inferences dominate the end-to-end latency. In addition, the monetary costs of LLM usage linearly increase with the number of tokens sent to and received from LLM. Q2 replaces the LLM-based movie summarization with RAG, which results in faster execution but increases the number of tokens compared to Q1. As shown in Fig. 16,

our CACTUSDB is capable of factorizing the nested LLM functions and pushing down the LLM summarization call and RAG call before the `cross_join` to avoid repeatedly LLM inferences on the same records, resulting in $3\times$ speed up and 72.4% reduction of tokens (and costs) on average. We noticed that even without enabling our reusable MCTS query optimizer, CACTUSDB benefits from Velox’s built-in runtime optimization, such as filter push-down. These runtime optimizations effectively reduced the total number of input/output tokens sent to the LLMs by 25% for Q1 and 59% for Q2 on average with end-to-end latency speedups of $1.1\times$ to $2.1\times$.

I. Comparison of Optimization Algorithms on Recommendation Queries

TABLE IV: Comparison of Optimization Algorithms (unit: second)

	Recommendation-Q1			Recommendation-Q2			Recommendation-Q3		
	Opt.	Exec.	Total	Opt.	Exec.	Total	Opt.	Exec.	Total
Un-optimized	NA	144.2	144.2	NA	13.6	13.6	NA	36.5	36.5
Arbitrary	0.9	1.8	2.7	5.1	131.3	136.3	4.4	23.7	28.1
Heuristic	0.2	5.2	5.4	0.3	12.3	12.6	0.1	16.3	16.3
Vanilla MCTS	5.2	0.3	5.5	6.8	6.8	13.6	10.3	8.7	18.9
Reusable MCTS-two models	1.5	0.2	1.7	2.1	6.5	8.6	3.6	8.8	12.4

Recommendation queries. We break down the end-to-end latency of each query into two parts: query optimization latency and query execution latency. Tab. IV compares the latency for query optimization (Opt.), query execution (Exec.), and the total of them. **Our reusable MCTS strikes the best trade-off between overhead and effectiveness compared to other baselines. Even though it introduces additional overhead for query embedding, the query optimization latency is greatly saved by reusing the MCTS.** The arbitrary optimization strategy could lead to better or worse query execution time compared to unoptimized queries. That is because not all optimization rules will be beneficial. Heuristic optimization strategy will always lead to better efficiency (optimization latency). However, simply relying on pre-defined heuristics cannot exploit the best optimization techniques for all queries. Vanilla MCTS demonstrates effectiveness in searching for optimal query plans due to its query-specific optimization. However, it requires building an MCTS search tree from scratch every time, which worsens the optimization latency.

J. Synthetic Model Sampling

We design a model sampling strategy to generate diverse architectures for collecting the training data for the Model2Vec model. The strategy is built on a set of reusable model-building blocks, which we denote as parameterized operators:

- ConvolutionalLayer (in, out, kernel, stride, padding)
- PoolingLayer (kernel, stride, padding)
- LinearLayer (in, out, activation, bias)
- BatchNormalization (num_features, ϵ)
- Dropout(p)
- EmbeddingLayer (num_tuples, dim)
- SVD (dim)

- Decision_Forest (n_{trees} , d_{max})
- DecisionTree (d_{max})
- Activation $\in \{\text{ReLU}, \text{Sigmoid}, \text{Tanh}, \text{Softmax}\}$
- Concat ($\cdot$)
- CosineSimilarity ($\cdot, \cdot$)
- Flatten ($\cdot$)

Each operator can be parameterized within a configurable range, and blocks may be repeated between a minimum and maximum depth. Using these primitives, users can compose customized model templates based on their needs. In our work, we define a set of representative model templates, which are used to generate training data, as illustrated in List. 4:

```

1 // Multi-Layer Perceptron (MLP)
2 MLP :=
3   Repeat{min_repeat, max_repeat}{
4     LinearLayer(RANGE(in_min, in_max),
5                 RANGE(out_min, out_max),
6                 CHOICE(Activation))
7   }
8
9 // Two-Tower Model
10 Tower :=
11   Repeat{min_repeat, max_repeat}{
12     LinearLayer(RANGE(in_min, in_max),
13                 RANGE(out_min, out_max),
14                 CHOICE(Activation))
15   }
16 TwoTowerModel := CosineSimilarity(Tower, Tower)
17
18 // Deep Learning Recommendation Model (DLRM-
19 style)
20 DLRM :=
21   Concat(
22     Repeat{min_repeat, max_repeat}{
23       EmbeddingLayer(RANGE(num_min, num_max),
24                       RANGE(dim_min, dim_max))
25     },
26     Repeat{min_repeat, max_repeat}{
27       LinearLayer(RANGE(in_min, in_max),
28                   RANGE(out_min, out_max),
29                   CHOICE(Activation))
30     }
31   )
32   -> Repeat{min_repeat, max_repeat}{
33     LinearLayer(RANGE(in_min, in_max),
34                 RANGE(out_min, out_max),
35                 CHOICE(Activation))
36   }
37
38 // Convolutional Neural Network (CNN)
39 CNN :=
40   Repeat{min_repeat, max_repeat}{
41     ConvolutionalLayer(RANGE(in_min, in_max),
42                        RANGE(out_min, out_max),
43                        RANGE(kernel_min,
44                              kernel_max))
45     -> BatchNorm(num_features)
46     -> CHOICE(Activation)
47     -> PoolingLayer(RANGE(kernel_min,
48                             kernel_max))
49   }
50   -> Flatten()

```

```

48 -> Repeat{min_repeat, max_repeat}{
49   LinearLayer(RANGE(in_min, in_max),
50               RANGE(out_min, out_max),
51               CHOICE(Activation))
52 }
53
54 // Decision Forest
55 DecisionForest :=
56   DecisionForest(RANGE(n_trees_min,
57                        n_trees_max),
58                 RANGE(depth_min, depth_max))

```

Listing 4: Predefined model templates

In practice, when sampling a model, the system randomly selects one of these templates and instantiates its building blocks by drawing parameter values (e.g., layer widths, kernel sizes, embedding dimensions, number of trees) from the specified ranges. This process yields a rich variety of architectures, while ensuring that the sampled models remain consistent with patterns observed in real-world in-database ML workloads.

K. Synthetic Query Templates

We describe the query templates used to generate training data and evaluation queries. In the MovieLens dataset, we convert the three queries described in Sec. F into templates and pick the seven most voted data science pipelines from Kaggle [100]. In the TPCx-AI [39] dataset, in addition to the tree queries we described in Sec G, we convert the other use cases queries into templates as illustrated in List. 5 and List. 6. The query sampler randomly samples a model architecture and configuration parameters, and randomly samples filters with different selectivity.

```

1 // Template 4: User Rating Prediction for
2 Movies
3 SELECT DNN(process_gender(gender),
4            normalize_age(age), normalize_occupation(
5            occupation), split_genres(genres))
6 FROM user CROSS JOIN movie
7 [WHERE AGE_FILTER | GENDER_FILTER |
8    OCCUPATION_FILTER | GENRE_FILTER];
9
10 // Template 5: User Opinion Prediction (
11 Negative/Neutral/Positive)
12 SELECT DNN(user_id, movie_id)
13 FROM user
14 [WHERE AGE_FILTER | GENDER_FILTER |
15    OCCUPATION_FILTER | ZIP_CODE_FILTER];
16
17 // Template 6: Recommend Movies to Users via
18 SVD
19 SELECT SVD(user_id, movie_id)
20 FROM user CROSS JOIN movie
21 [WHERE AGE_FILTER | GENDER_FILTER |
22    OCCUPATION_FILTER | ZIP_CODE_FILTER |
23    GENRE_FILTER];
24
25 // Template 7: Rating Prediction with
26 Collaborative Filtering
27 SELECT LightFM(user_id, movie_id)
28 FROM user CROSS JOIN movie
29 [WHERE AGE_FILTER | GENDER_FILTER |
30    OCCUPATION_FILTER | ZIP_CODE_FILTER |
31    GENRE_FILTER];

```

```

20
21 // Template 8: Rating Prediction with
    AutoEncoder
22 SELECT AutoEncoder(user_id, movie_id)
23 FROM user
24 [WHERE AGE_FILTER | GENDER_FILTER |
    OCCUPATION_FILTER | ZIP_CODE_FILTER];
25
26 // Template 9: Detect Gender Sterotypes with
    DNN
27 SELECT DNN(prepare_feature(...))
28 FROM ratings JOIN movie ON ratings.movie_id =
    movie.movie_id
29 [WHERE GENRE_FILTER | TIMESTAMP_FILTER];
30
31 // Template 10: Rating Prediction
32 SELECT DNN(movie_id, age, occupation)
33 FROM user CROSS JOIN movie
34 [WHERE AGE_FILTER | GENDER_FILTER |
    OCCUPATION_FILTER | ZIP_CODE_FILTER |
    GENRE_FILTER];

```

Listing 5: Query templates for MovieLens dataset

```

1 // Template 4: Product Rating Prediction with
    SVD
2 SELECT SVD(user_id, product_id)
3 FROM product_rating
4 JOIN product ON productID = p_product_id
5 JOIN customer ON c_customer_sk = userID
6 [WHERE PRODUCT_DEPARTMENT_FILTER |
    CUSTOMER_BIRTH_DATE_FILTER |
    CUSTOMER_BIRTH_COUNTRY_FILTER];
7
8 // Template 5: Spam Review Detection with DNN
9 SELECT DNN(tf_idf_features(text))
10 FROM review_comments
11 [WHERE ID_FILTER];
12
13 // Template 6: Classification of Trips with
    DNN/Decision Forest
14 SELECT DNN/DecisionForest(prepare_feature(...)
    )
15 FROM (
16     SELECT o_order_id, department, quantity,
17           SUM(quantity) AS scan_count,
18           MIN(EXTRACT(DOW FROM date)) AS
                weekday
19     FROM tpcxai_order_serving
20     JOIN tpcxai_lineitem_serving ON o_order_id =
        li_order_id
21     JOIN tpcxai_product_serving ON li_product_id
        = p_product_id
22     GROUP BY o_order_id, date, department,
        quantity
23 )
24 [WHERE TIMESTAMP_FILTER | DEPARTMENT_FILTER |
    WEEKDAY_FILTER | PRICE_FILTER |
    QUANTITY_FILTER];
25
26 // Template 7: Fraud Detection
27 SELECT DNN/LogisticRegression(concat(EXTRACT(
    HOUR FROM time)/23, amount/
    transaction_limit))
28 FROM financial_transactions
29 JOIN financial_account ON sender_id =
    fa_customer_sk

```

```

30 JOIN customer ON fa_customer_sk =
    c_customer_sk
31 [WHERE TIMESTAMP_FILTER | AMOUNT_FILTER |
    CUSTOMER_BIRTH_INFO_FILTER];
32
33 // Template 8: Sales Prediction
34 SELECT DNN(store_id_encoder(store_id),
    department_encoder(department),
    normalize_week(week))
35 FROM store_dept
36 [WHERE DEPARTMENT_FILTER | WEEK_FILTER |
    STORE_FILTER];
37
38 // Template 9: Customer Segmentation
39 SELECT KMeans/DNN(prepare_feature(...))
40 FROM (
41     SELECT YEAR(date) AS invoice_year,
42           quantity * price AS row_price,
43           or_return_quantity * price AS
                return_row_price
44     FROM order
45     JOIN lineitem ON o_order_id = li_order_id
46     JOIN order_returns ON or_order_id =
        o_order_id
47     JOIN product ON p_product_id = li_product_id
48 )
49 [WHERE DATE_FILTER | STORE_FILTER |
    PRODUCT_FILTER | DEPARTMENT_FILTER];
50
51 // Template 10: Customer Satisfaction
    Prediction
52 SELECT DNN(prepare_feature(...))
53 FROM customer
54 CROSS JOIN product
55 JOIN lineitem ON li_product_id = p_product_id
56 [WHERE CUSTOMER_INFO_FILTER |
    DEPARTMENT_FILTER | PRICE_FILTER];

```

Listing 6: Query templates For TPCx-AI dataset